Proyecto Final.

Gallardo Valdez Brayan Alexis Granados Rovira José Pablo
Lozano León Diego Márquez Corona Danna Lizette
Paredes González Emiliano Sebastián Tapia Anrubio Irving Axel

29 de Mayo de 2026

Índice

1. Introducción **4**

2. Análisis de requerimientos **4**

3. Consideraciones de diseño **4**
3.1. Sucursal y clínica . . . 5
3.2. Personal y especialización . . . 5
3.3. Medicamentos que preparan los farmacéuticos . . . 6
3.4. Insumos . . . 6
3.5. Clientes y tipos de cliente . . . 6
3.6. Regla de descuentos . . . 7
3.7. Consultas médicas . . . 7
3.8. Recetas médicas . . . 7
3.9. Medicamentos . . . 8
3.10. Proveedores . . . 8
3.11. Tickets . . . 9
3.12. Control de inventario . . . 9
3.13. Compra en línea . . . 10
3.14. Decisiones generales de integridad . . . 10

4. Modelo Entidad–Relación **10**

5. Modelo Relacional **11**

6. Dependencias funcionales **12**

7. Implementación física en PostgreSQL **22**
7.1. Estrategia de creación y limpieza del esquema . . . 22
7.2. Selección de tipos de dato . . . 22
7.3. Integridad de entidad: llaves primarias . . . 22
7.4. Integridad de dominio: NOT NULL, CHECK, UNIQUE y DEFAULT . . . 23
7.5. Integridad referencial: llaves foráneas y acciones referenciales . . . 24
7.6. Documentación del esquema y orden de carga . . . 24

8. Poblamiento de la base de datos **24**

9. Procedimientos, funciones y disparadores **26**
9.1. Funciones (archivo `Funciones.sql`) . . . 26
9.2. Procedimientos almacenados (archivo `SP.sql`) . . . 27
9.3. Archivo de triggers . . . 27
9.4. Importancia de los triggers . . . 28
9.5. Control del stock de medicamentos . . . 28
9.6. Cálculo de descuentos . . . 29
9.7. Cálculo del precio bruto y precio neto . . . 30
9.8. Eventos que detectan los triggers . . . 30
9.9. Relación con la integridad de los datos . . . 31
9.10. Alcance del archivo . . . 31

10. Consultas **32**
10.1. Consulta xiv — Distribución de sucursales por estado 32
10.2. Consulta xv — Sucursales con clínica y capacidad médica 32
10.3. Consulta xvi — Cantidad de empleados por sucursal 33
10.4. Consulta xvii — Salarios promedio por categoría de empleado 34
10.5. Consulta xviii — Farmacéuticos y preparación de medicamentos especiales 34
10.6. Consulta xix — Métodos de pago utilizados por clientes 35
10.7. Consulta xx — Tickets con descuentos o promociones 36
10.8. Consulta xxi — Número de consultas médicas realizadas 36
10.9. Consulta xxii — Médicos activos y productividad 37
10.10. Consulta xxiii — Recetas relacionadas con alergias 38
10.11. Consulta xxiv — Stock e inventario por sucursal 39
10.12. Consulta xxv — Medicamentos próximos a caducar 40
10.13. Consulta xxvi — Proveedores de medicamentos e insumos 41
10.14. Consulta xxvii — Ventas presenciales vs ventas en línea 41
10.15. Consulta xxviii — Comparación entre ingresos médicos y ventas de medicamentos 42

11. Conclusión **43**

1. Introducción

El proyecto se basa en el caso de uso *Una farmacia de otro mundo*, donde se busca diseñar una base de datos para administrar mejor la información de una cadena de farmacias. La idea principal es dejar de depender de registros físicos, hojas de cálculo o información separada, y pasar a un sistema más organizado que permita manejar sucursales, clientes, personal, proveedores, medicamentos, insumos, consultas médicas, recetas y tickets.

En nuestro caso, el proyecto fue trabajado a partir de las prácticas realizadas durante el semestre. Primero se identificaron las entidades principales del problema y después se fueron corrigiendo los diagramas y scripts conforme avanzaban las entregas.

La base de datos fue pensada para que una farmacia pueda consultar y registrar información importante de su operación diaria. Por ejemplo, permite saber qué sucursales existen, qué personal trabaja en cada una, qué medicamentos o insumos se manejan, qué proveedores los suministran, qué clientes están registrados y cómo se relacionan las consultas médicas con sus recetas y tickets.

2. Análisis de requerimientos

A partir del caso de uso se identificó que el sistema debía guardar información de varias áreas de la farmacia. No se trataba solamente de registrar productos, sino de representar el funcionamiento completo de la cadena: sucursales, clínicas, empleados, clientes, medicamentos, proveedores, consultas y ventas.

Uno de los requerimientos principales fue registrar las sucursales de la farmacia. Para esto se consideraron datos como el nombre, dirección y teléfonos. Algunas sucursales también pueden tener una clínica integrada, por lo que se incluyó información específica de clínicas, como nombre, número de cuartos y horarios.

También se necesitó guardar información del personal. En el diseño se tomó una entidad general llamada `PERSONAL`, de la cual se desprenden distintos tipos de trabajadores, como médicos, enfermeras, cajeros, personal de limpieza, cuidadores y farmacéuticos. Esto ayudó a evitar repetir datos comunes, como nombre, RFC, domicilio, salario y sucursal donde trabaja cada persona.

Otro punto importante fue el registro de clientes. En el modelo se consideró que un cliente puede ser físico, cliente en línea o paciente. Por eso, en la tabla `CLIENTE` se agregaron atributos que permiten distinguir estos casos, además de datos personales como nombre, fecha de nacimiento, domicilio, método de pago, usuario y contraseña para compras en línea.

Para la parte médica, el sistema contempla consultas, médicos, enfermeras y recetas médicas. Cada consulta se relaciona con un cliente, un médico, una enfermera, una clínica y un ticket. Además, si la consulta genera una receta, esta se relaciona mediante una tabla especial que permite representar esa conexión.

En cuanto a medicamentos e insumos, se identificó la necesidad de registrar tanto los productos que se venden como los insumos utilizados para preparar medicamentos. También se consideraron los proveedores, ya que un mismo medicamento o insumo puede ser suministrado por más de un proveedor. Para representar esta relación, se usaron tablas intermedias como `PROVEER_MEDICAMENTO` y `PROVEER_INSUMO`.

Al final, también se consideraron tickets y compras. Un ticket se relaciona con una sucursal y un cliente, mientras que los medicamentos comprados se registran en la tabla `COMPRAR`. Así se puede saber qué cliente compró, en qué sucursal y qué medicamentos estuvieron involucrados.

3. Consideraciones de diseño

En esta sección se documentan y justifican las decisiones de diseño más relevantes tomadas durante la construcción del modelo, comparando los requerimientos del caso de uso con lo que finalmente se

implementó. Se recorren las entidades y reglas del negocio una por una, señalando dónde el modelo cubre fielmente lo solicitado y dónde se simplificó o se tomó una interpretación distinta.

3.1. Sucursal y clínica

El caso de uso solicita guardar para las sucursales: nombre, dirección completa (calle, número interior, número exterior, colonia y estado), teléfono y horarios de atención. Para las clínicas integradas pide además: nombre de la clínica, número de cuartos, número de empleados y horarios de atención.

La tabla `SUCURSAL` recoge el nombre y la dirección completa solicitados. El teléfono se modeló como atributo multivaluado en la tabla `TELEFONO_SUCURSAL`, lo que permite registrar más de un número por sucursal sin alterar la estructura principal. Los horarios de atención de las clínicas se modelaron de la misma forma en `HORARIO_CLINICA` pues pueden ser varios horarios (fines de semana, entre semana, etc.)

La primera decisión de diseño relevante fue **omitir el atributo número de empleados** de la tabla `CLINICA`. Este dato es un valor derivado: puede obtenerse en cualquier momento contando los registros de `PERSONAL` asociados a la sucursal que contiene la clínica. Almacenarlo de forma fija implicaría redundancia y el riesgo de que quedara desactualizado cada vez que el personal cambia. En su lugar, este valor se calcula dinámicamente cuando se necesita por medio de nuestros triggers explicados en el reporte general.

La relación entre sucursal y clínica se modeló con la llave foránea `CLINICA.IDSucursal` que apunta a `SUCURSAL.IDSucursal` con la acción `ON DELETE RESTRICT`. De esta manera no es posible eliminar una sucursal mientras tenga clínicas registradas, lo que protege la estructura operativa de la cadena. Esto, pues no tiene sentido que la sucursal deje de existir mientras ésta tenga clínicas registradas.

3.2. Personal y especialización

El caso de uso establece varios tipos de personal: farmacéuticos, cajeros, personal de limpieza, cuidadores y, en sucursales con clínica, médicos y enfermeras. Para todo el personal se requieren los mismos datos base: nombre completo, cédula profesional, RFC, teléfonos, correos electrónicos, horario de trabajo, domicilio y salario.

Como se vio en clase, los atributos se modelaron de la siguiente manera: La tabla `PERSONAL` almacena todos los atributos compartidos por cualquier trabajador, mientras que seis tablas especializadas (`MEDICO`, `ENFERMERA`, `CAJERO`, `LIMPIEZA`, `CUIDADOR` y `FARMACEUTICO`) extienden esa información con los atributos propios de cada rol. Cada tabla especializada usa `IDPersonal` simultáneamente como llave primaria y como llave foránea hacia `PERSONAL`, estableciendo una relación de uno a uno.

Los teléfonos, correos y horarios del personal son atributos multivaluados en el caso de uso. Se modelaron en tablas independientes como se vio en clase, (`TELEFONO_PERSONAL`, `CORREO_PERSONAL` y `HORARIO_PERSONAL`), cada una con llave primaria compuesta por `IDPersonal` más el valor del atributo. Para los **médicos**, el caso de uso pide especialidad, institución de egreso y vigencia de certificación. Los tres atributos están en la tabla `MEDICO`. Adicionalmente se agregó una restricción `CHECK` que exige que la vigencia de certificación sea igual o posterior a la fecha actual, evitando registrar médicos con certificación vencida, esto es una decisión de diseño importante para que los datos sean congruentes con los médicos que se tienen o que se intentan contratar.

Para las **enfermeras**, el caso de uso pide el tipo de procedimiento a cargo y si poseen certificación de reanimación. Ambos atributos se representan en la tabla `ENFERMERA` mediante `TipoProcedimiento` y `CertificadoReanimacion`.

3.3. Medicamentos que preparan los farmacéuticos

El caso de uso describe cinco categorías de medicamentos que los farmacéuticos pueden preparar: formulaciones, preparados oficiales, medicamentos estériles, preparaciones pediátricas y preparaciones dermatológicas.

Se decidió modelar estas categorías como **atributos booleanos** dentro de la tabla `MEDICAMENTO`: `MedicamentosEsteriles`, `PreparadosOficiales`, `Pediatrica` y `Dermatologica`. Adicionalmente se incluyeron los campos `Preparaciones` y `Formulacion` de tipo `TEXT` para describir libremente el contenido de las formulaciones.

Esta decisión simplifica notablemente las consultas de filtrado por categoría, ya que basta evaluar una bandera booleana en lugar de hacer combinaciones con tablas adicionales. La contraparte es que un medicamento puede pertenecer a varias categorías a la vez, lo cual el modelo permite de forma intencional. Si en algún momento el negocio requiriera que cada medicamento perteneciera a exactamente una categoría, la solución más apropiada sería una tabla de catálogo de categorías relacionada por llave foránea.

3.4. Insumos

El caso de uso plantea varios atributos para los insumos: nombre científico, nombre comercial, tipo, forma física, potencia, grado farmacopeico, riesgo, si es estéril, temperatura de almacenamiento, sensibilidad y observaciones.

La implementación cubrió los atributos que coinciden con la información farmacéutica de uso más frecuente, agrupándolos en la tabla `INSUMO` con los campos: `NombreCientifico` (que además funciona como llave primaria natural), `Presentacion`, `FormaFarmaceutica`, `Concentracion`, `ViaAdministracion`, `Clasificacion`, `Descripcion`, `LaboratorioFabricante`, `NombreComercial`, `TipoDeControl`, `PrecioPublico` y `PrecioUnitario`.

Los atributos más especializados de la especificación (grado farmacopeico, nivel de riesgo, condición de esterilidad, temperatura de almacenamiento, sensibilidad y observaciones) **no se modelaron como columnas independientes**. La decisión de diseño fue que el campo `Descripcion`, de tipo `TEXT`, puede absorber esa información de forma narrativa, lo que mantiene el esquema más simple y evita una gran cantidad de columnas opcionales con valores nulos. Es una simplificación válida para el alcance del proyecto, además de facilitar la obtención de conocimiento teniendo menos datos null con los que después se tendrá que trabajar.

La elección de `NombreCientifico` como llave primaria natural es adecuada en el contexto farmacológico, donde ese nombre identifica de forma única a un insumo. La consecuencia es que si el nombre cambiara, ese cambio debe propagarse a todas las tablas que lo referencian; el esquema lo maneja mediante la acción `ON UPDATE CASCADE` en las llaves foráneas correspondientes.

3.5. Clientes y tipos de cliente

El caso de uso pide registrar a los clientes con nombre completo, teléfonos, correos, fecha de nacimiento, domicilio y método de pago. Además distingue entre clientes que compran en sucursal (físicos), clientes que compran en línea (con usuario, contraseña, número de tarjeta y fecha de vencimiento) y pacientes que reciben consultas.

La decisión de diseño fue **unificar los tres tipos en una sola tabla** `CLIENTE` con tres indicadores booleanos: `EsClienteFisico`, `EsClienteEnLinea` y `EsPaciente`. Los atributos exclusivos del cliente en línea (`Usuario`, `Contrasena`, `NumeroTarjeta` y `VencimientoTarjeta`) quedaron como columnas opcionales que solo se llenan cuando el cliente es en línea.

Esta decisión simplifica las consultas que necesitan relacionar a los clientes con sus tickets, consultas o recetas, ya que toda la información del cliente reside en una sola tabla. La alternativa habría sido crear tres tablas especializadas, como se hizo con el personal, pero se consideró que la superposición entre tipos (un mismo cliente puede ser físico, en línea y paciente a la vez) hacía más práctica la solución de banderas booleanas.

Los teléfonos y correos del cliente se modelaron como atributos multivaluados en `TELEFONO_CLIENTE` y `CORREO_CLIENTE`, siguiendo el mismo criterio aplicado al personal.

3.6. Regla de descuentos

El caso de uso establece una regla de descuentos basada en la frecuencia de visitas de cada cliente: si acude más de 6 veces al año se le aplica un 25 %, si acude 4 veces un 10 % y si acude 2 veces un 5 %.

La implementación adoptó una **interpretación distinta** para el cálculo del descuento. En lugar de contar las visitas de cada cliente, el descuento se determina según el número total de tickets registrados en el sistema durante el año en curso. Los rangos implementados son: de 0 a 99 tickets, 0 % de descuento; de 100 a 499 tickets, 5 %; de 500 a 1999 tickets, 10 %; y de 2000 tickets o más, 15 %.

Esta diferencia respecto al caso de uso es importante y debe quedar explícita. El caso de uso plantea un descuento *personalizado por cliente*, en función de su historial individual de visitas; la implementación aplica un descuento *global*, en función del volumen total de operaciones de la cadena durante el año. Ambos son esquemas de descuentos válidos. Esta decisión se originó en el desarrollo de las prácticas del semestre y se conservó en el proyecto final por consistencia con el resto de los componentes ya construidos sobre ella, en particular los disparadores que calculan los importes del ticket. Además de que es mucho más sensato generar descuento a partir de las veces que compras en la tienda que de las veces que te paras ahí sin generar nada a La Farmacia de Otro Mundo.

3.7. Consultas médicas

El caso de uso pide guardar para cada consulta: la fecha y hora en que se realizó, el médico que la realizó, la enfermera que apoyó, el cliente que fue atendido y el diagnóstico. También indica que, cuando el cliente pague la consulta, debe guardarse su precio.

La tabla `CONSULTA` incluye todos estos atributos. El precio se almacena en `CostoConsulta`, con una restricción `CHECK` que impide valores negativos. La enfermera se modeló como participación opcional: `IDEnfermera` admite nulos y su llave foránea usa la acción `ON DELETE SET NULL`, en coherencia con la descripción del caso de uso, que presenta a la enfermera como un apoyo que puede o no estar presente.

Una decisión de diseño relevante fue **ligar cada consulta a un ticket** mediante `CONSULTA.IDTicket` y forzar que esa relación sea uno a uno con una restricción `UNIQUE` sobre `IDTicket`. Esto modela la regla de que una consulta genera exactamente un cobro, registrado en el ticket correspondiente, tal como lo plantea la especificación cuando indica que la consulta debe pagarse en la sucursal.

Las llaves foráneas de `CONSULTA` apuntan a las tablas `MEDICO` y `ENFERMERA`, no directamente a `PERSONAL`. Con ello se garantiza, a nivel de base de datos, que solo personal con el rol adecuado pueda quedar registrado como responsable o apoyo de una consulta.

3.8. Recetas médicas

El caso de uso pide guardar en las recetas una lista amplia de datos: nombre del paciente, edad, fecha de nacimiento, peso, talla, número de receta, identificador del paciente, alergias reportadas, diagnóstico, nombre del médico, cédula profesional del médico, especialidad, consultorio, turno, los medicamentos recetados, su forma farmacéutica, concentración, presentación, dosis prescrita, frecuencia, vía de administración y duración o número de días.

La implementación distribuyó esta información en dos tablas: `RECETA_MEDICA` para el encabezado de la receta y `PEDIR` para el detalle de los medicamentos.

La tabla `RECETA_MEDICA` almacena `FechaNacimiento`, `Peso`, `Talla`, `Alergias`, `Diagnostico`, `Consultorio` y `Turno`. Sobre ella se aplicaron varias decisiones de simplificación, todas orientadas a evitar redundancia:

- El **nombre del paciente** no se almacena en la receta, porque puede obtenerse con la relación que une la receta con su consulta y la consulta con el cliente. Guardarlo en la receta sería duplicar un dato que ya existe.
- El **nombre y la cédula del médico** tampoco se almacenan en la receta, por la misma razón: se obtienen a través de la consulta y de la especialización `MEDICO`. La especificación los menciona porque en una receta impresa deben aparecer físicamente, pero en la base de datos preferimos simplemente referenciarlos en lugar de duplicarlos.
- La **edad** del paciente no se guarda como columna, ya que es un valor derivado de la fecha de nacimiento; se calcula cuando se necesita por medio de nuestros triggers.
- La **duración o número de días** del tratamiento no se incluyó como columna independiente en `PEDIR`. Esta información puede incorporarse dentro de los campos `Frecuencia` o `Dosis` como texto, lo que es una simplificación razonable para el alcance del proyecto.

La tabla `PEDIR` representa la relación muchos a muchos entre una receta y los medicamentos que indica, e incluye `Dosis` y `Frecuencia` como atributos propios de esa relación. Los atributos de forma farmacéutica, concentración, presentación y vía de administración no se repiten en `PEDIR` porque ya están disponibles en las tablas de los productos referenciados.

La relación entre consulta y receta se implementó mediante la tabla `GENERAR_CONSULTA_RECETA`, con restricciones `UNIQUE` sobre ambas llaves foráneas. Esto garantiza la cardinalidad uno a uno: una consulta genera a lo más una receta y una receta proviene de a lo más una consulta.

3.9. Medicamentos

El caso de uso pide para los medicamentos: nombre comercial, nombre genérico, forma farmacéutica, concentración o potencia, presentación, vía de administración, clasificación, tipo de control, laboratorio fabricante, descripción y stock disponible.

La tabla `MEDICAMENTO` cubre el stock, los precios público y unitario, y las categorías de preparación especial descritas anteriormente. Sin embargo, varios atributos puramente descriptivos (nombre comercial, nombre genérico, forma farmacéutica, concentración, presentación, vía de administración, clasificación, tipo de control, laboratorio y descripción) **no se modelaron como columnas independientes en MEDICAMENTO**.

La razón de esta decisión es que, en el modelo implementado, los medicamentos que vende la farmacia y los insumos con los que se preparan comparten una estructura descriptiva muy parecida. Esa estructura descriptiva se concentró en la tabla `INSUMO`, y la relación `UTILIZAR` conecta cada medicamento con los insumos que lo componen. De este modo se evita duplicar la misma estructura descriptiva en dos tablas distintas.

3.10. Proveedores

El caso de uso pide para los proveedores: número, razón social, los medicamentos e insumos que provee, dirección y teléfonos. También establece de forma expresa que un mismo medicamento o insumo puede ser proporcionado por más de un proveedor, y que para cada suministro deben conocerse la fecha de recibo, la fecha de caducidad, las condiciones de almacenamiento, el precio unitario y el precio público.

La tabla `PROVEEDOR` cubre la razón social y la dirección completa. El teléfono se modeló como atributo multivaluado en `TELEFONO_PROVEEDOR`.

La relación entre proveedor, medicamento y sucursal se modeló como una **relación ternaria** en la tabla `PROVEER_MEDICAMENTO`, que incluye, además de los tres participantes, la condición de almacenamiento, la cantidad, la fecha de recibo y la fecha de caducidad. Esta estructura permite registrar que un mismo proveedor entrega distintos medicamentos a distintas sucursales en distintas fechas. Para distinguir cada entrega (cada lote), la llave primaria compuesta incluye la fecha de recibo.

La misma estructura ternaria se repite en `PROVEER_INSUMO` para los insumos.

Esta decisión satisface directamente el requerimiento de que un medicamento pueda tener varios proveedores, y como beneficio adicional permite rastrear el historial de entregas por sucursal, lo que resulta valioso para el control del inventario y la gestión de caducidades.

3.11. Tickets

El caso de uso indica que en el ticket debe aparecer: la sucursal donde se realizó la compra, los medicamentos comprados (si los hubo), la consulta realizada (si la hubo), el precio bruto y el precio neto después de aplicar descuentos.

La tabla `TICKET` contiene la sucursal, el cliente, la fecha, la hora, el precio bruto, el precio neto y el descuento aplicado. Los medicamentos comprados se registran en la tabla `COMPRAR`, y la consulta asociada se vincula a través de `CONSULTA.IDTicket`.

El precio bruto y el precio neto se mantienen actualizados de forma automática mediante los disparadores definidos en el proyecto, de manera que siempre reflejan el estado actual de los productos asociados al ticket sin necesidad de cálculos manuales.

Una consideración técnica que conviene dejar registrada es que el precio neto, dentro de los disparadores, se calcula aplicando el porcentaje de descuento almacenado en el ticket, sin incluir IVA. En contraste, algunas consultas de análisis del proyecto calculan un precio neto aplicando un IVA del 16 % como criterio alternativo. Ambos cálculos son válidos en sus respectivos contextos (uno es el importe que efectivamente se almacena en el ticket y el otro es un cálculo analítico para una consulta específica), pero es importante tenerlo presente al interpretar los resultados para no confundir ambas cifras.

3.12. Control de inventario

El caso de uso establece que debe conocerse el stock disponible de cada medicamento, con el fin de poder solicitar reabastecimiento a los proveedores cuando sea necesario.

La columna `Stock` de la tabla `MEDICAMENTO` cumple esa función, y su valor se mantiene actualizado de forma automática mediante tres disparadores que reaccionan a los movimientos de inventario:

- Cuando un proveedor entrega medicamento, el stock aumenta.
- Cuando un farmacéutico prepara medicamento, el stock aumenta.
- Cuando un cliente realiza una compra, el stock disminuye, previa validación que impide que el inventario quede en valores negativos.

Los tres disparadores contemplan también las operaciones de actualización y eliminación, revirtiendo el efecto del valor anterior antes de aplicar el nuevo. Con ello el inventario permanece exacto incluso ante correcciones de captura o cancelaciones.

3.13. Compra en línea

El caso de uso establece que los clientes pueden comprar medicamentos o insumos en línea, siempre que no se necesite receta; que para ello deben iniciar sesión con usuario y contraseña; que al realizar una compra en línea se guardan el número de tarjeta y su fecha de vencimiento para futuras compras; y que esa operación también genera un ticket, equivalente al de una compra presencial.

En el modelo, los atributos `Usuario`, `Contrasena`, `NumeroTarjeta` y `VencimientoTarjeta` residen en la tabla `CLIENTE` como columnas opcionales que se utilizan cuando el cliente es en línea. El ticket no distingue estructuralmente entre una compra presencial y una en línea; la naturaleza de la venta se infiere a partir del tipo de cliente asociado al ticket. Esta decisión permite, sin embargo, clasificar y comparar ambas modalidades en las consultas de análisis del proyecto.

La restricción del caso de uso según la cual los productos comprados en línea no deben requerir receta **no se forzó a nivel de base de datos**: no existe una restricción que impida asociar a un ticket en línea un medicamento de venta bajo receta. Esa validación se asume responsabilidad de la capa de aplicación, lo que se documenta aquí como una decisión consciente y como un posible punto de refuerzo futuro.

3.14. Decisiones generales de integridad

Acciones referenciales diferenciadas. Las llaves foráneas no usan una sola política uniforme, sino que se eligió la acción adecuada según el significado de cada relación. Se utilizó borrado en cascada para los datos que no tienen sentido sin su entidad principal (como los teléfonos y correos de un cliente, o las especializaciones de un trabajador) y borrado restringido para las entidades cuya eliminación pondría en riesgo información histórica valiosa (como una sucursal con personal, o un medicamento con compras registradas). Para la participación opcional de la enfermera en una consulta se utilizó la acción de poner el valor en nulo.

Orden de carga deliberado. Se estableció que los disparadores se instalen antes de insertar los datos de prueba, con el fin de que su funcionamiento se verifique desde el inicio del poblamiento y de que los valores derivados (como el stock y los importes de los tickets) se calculen correctamente desde la primera inserción.

El conjunto de decisiones aquí documentadas refleja un equilibrio entre dos objetivos: por un lado, ser fiel a los requerimientos del caso de uso *Una farmacia de otro mundo* y, por otro, mantener un esquema claro, consistente y libre de redundancias innecesarias, además de facilitar la implementación de todo lo que pedía el cliente con un modelo efectivo que aterrice lo que el cliente pide con lo que nosotros podemos ofrecerle.

En la mayoría de las entidades el modelo cubre fielmente lo solicitado, y en varios puntos lo refuerza con restricciones de integridad que la especificación no exigía de forma explícita. Las diferencias más significativas respecto al caso de uso son dos: la regla de descuentos, que se implementó por volumen global de tickets en lugar de por visitas individuales del cliente (ya se explicó el por qué), y el control automático de inventario, que se desarrolló de forma más completa para medicamentos que para insumos. Ambas diferencias se documentan de manera transparente para que el lector comprenda el alcance real del sistema entregado.

4. Modelo Entidad–Relación

El modelo Entidad–Relación se desarrolló con el objetivo de representar de manera gráfica y conceptual el funcionamiento general de la cadena de farmacias descrita en el caso de uso. A partir del análisis de requerimientos y las consideraciones de diseño se identificaron las entidades principales, sus

atributos y las relaciones existentes entre ellas, buscando que el modelo reflejara las reglas de negocio más importantes del sistema.

Dentro del modelo se consideraron entidades principales como `CLIENTE`, `SUCURSAL`, `PERSONAL`, `MEDICAMENTO`, `INSUMO`, `PROVEEDOR`, `CONSULTA`, `RECETA_MEDICA`, `CLINICA` y `TICKET`. Estas entidades representan los componentes esenciales de la operación diaria de la farmacia y permiten organizar la información de forma estructurada.

Uno de los aspectos más importantes del modelo fue el manejo de la entidad `PERSONAL` como una superentidad. A partir de ella se derivan entidades especializadas como `MEDICO`, `ENFERMERA`, `FARMACEUTICO`, `CAJERO`, `LIMPIEZA` y `CUIDADOR`. Esta decisión permitió reutilizar atributos comunes como nombre, RFC, salario, domicilio y sucursal asignada, evitando redundancia y facilitando la administración de distintos tipos de empleados.

También se identificaron relaciones de distintos tipos. Por ejemplo, existen relaciones uno–a–muchos entre `SUCURSAL` y `PERSONAL`, ya que una sucursal puede tener muchos empleados, pero cada empleado pertenece únicamente a una sucursal. De forma similar, un cliente puede tener múltiples tickets o consultas médicas asociadas.

Además, se modelaron relaciones muchos–a–muchos mediante entidades intermedias. Un ejemplo es la relación entre `PROVEEDOR` y `MEDICAMENTO`, ya que un proveedor puede suministrar muchos medicamentos y un medicamento puede ser surtido por distintos proveedores. Para representar correctamente esta situación se utilizaron entidades asociativas como `PROVEER_MEDICAMENTO` y `PROVEER_INSUMO`.

En la parte médica, el modelo permite relacionar consultas con médicos, enfermeras, pacientes y recetas médicas. Esto permite representar el flujo completo de atención dentro de la clínica integrada a la farmacia. Asimismo, la relación entre `CONSULTA` y `RECETA_MEDICA` permite asociar recetas generadas durante una consulta específica.

Otro aspecto importante fue el manejo de atributos multivaluados, como teléfonos y correos electrónicos. En lugar de almacenar varios valores en una misma columna, se optó por modelarlos mediante entidades independientes relacionadas con clientes, sucursales o personal, siguiendo una estructura más normalizada.

En general, el modelo Entidad–Relación sirvió como base conceptual para posteriormente construir el modelo relacional y la implementación física en PostgreSQL. Gracias a este diseño fue posible representar de manera clara las entidades principales del sistema, sus relaciones y las restricciones necesarias para mantener la integridad de la información.

5. Modelo Relacional

Una vez definido el modelo Entidad–Relación, se realizó su traducción al modelo relacional con el objetivo de implementar la base de datos dentro del manejador PostgreSQL. Durante este proceso se transformaron las entidades en tablas, las relaciones en llaves foráneas y las relaciones muchos–a–muchos en tablas intermedias.

El diseño relacional se desarrolló buscando reducir redundancia de información, mantener integridad referencial y facilitar la ejecución de consultas sobre el sistema. Para ello se definieron llaves primarias en cada tabla y llaves foráneas para representar las relaciones existentes entre las entidades del modelo conceptual.

Las entidades principales del sistema se implementaron como tablas independientes, entre ellas `CLIENTE`, `SUCURSAL`, `PERSONAL`, `MEDICAMENTO`, `INSUMO`, `CONSULTA`, `RECETA_MEDICA`, `PROVEEDOR` y `TICKET`.

Las especializaciones de la entidad `PERSONAL`, como `MEDICO`, `ENFERMERA`, `FARMACEUTICO`, `CAJERO`, `LIMPIEZA` y `CUIDADOR`, se implementaron mediante tablas separadas que reutilizan la llave primaria de la superentidad. Esto permitió conservar una relación uno–a–uno entre cada empleado y su especialización correspondiente.

Asimismo, las relaciones muchos–a–muchos se implementaron mediante tablas intermedias. Por ejemplo, las relaciones entre proveedores y medicamentos, o entre tickets y medicamentos comprados, se representaron mediante tablas como `PROVEER_MEDICAMENTO`, `PROVEER_INSUMO` y `COMPRAR`.

También se implementaron tablas para atributos multivaluados, como teléfonos, horarios y correos electrónicos, permitiendo almacenar múltiples valores asociados a una misma entidad sin duplicar información.

Finalmente, el modelo relacional incorporó restricciones de integridad como llaves primarias, llaves foráneas, restricciones `NOT NULL`, `CHECK`, `UNIQUE` y valores por defecto, con el objetivo de asegurar consistencia y validez en los datos almacenados.

6. Dependencias funcionales

A continuación se presentan las dependencias funcionales de la base de datos, obtenidas principalmente del DDL, ya que las dependencias funcionales deben salir de las restricciones del esquema: llaves primarias, llaves únicas y reglas estructurales.

El propio DDL indica que las entidades fuertes tienen `PRIMARY KEY`, que las especializaciones usan `IDPersonal`, y que las relaciones multivaluadas, muchos a muchos y ternarias usan llaves compuestas. Además, se declaran restricciones `UNIQUE` como `Usuario`, `RFC`, `IDTicket` en `CONSULTA`, e identificadores únicos en `GENERAR_CONSULTA_RECETA`.

1. CLIENTE

$$\texttt{CLIENTE}(IDCliente, Nombre, ApellidoPaterno, ApellidoMaterno, FechaNacimiento, Calle, NumExterior, NumInterior, Colonia, Estado, MetodoPago, NumeroTarjeta, VencimientoTarjeta, Usuario, Contrasena, EsClienteEnLinea, EsClienteFisico, EsPaciente)$$

Llave primaria:

$$IDCliente$$

Dependencia funcional principal:

$$IDCliente \rightarrow Nombre, ApellidoPaterno, ApellidoMaterno, FechaNacimiento, Calle, NumExterior, NumInterior, Colonia, Estado, MetodoPago, NumeroTarjeta, VencimientoTarjeta, Usuario, Contrasena, EsClienteEnLinea, EsClienteFisico, EsPaciente$$

2. SUCURSAL

$$\texttt{SUCURSAL}(IDSucursal, Nombre, Calle, NumExterior, NumInterior, Colonia, Estado)$$

Llave primaria:

$$IDSucursal$$

Dependencia funcional:

$$IDSucursal \rightarrow Nombre, Calle, NumExterior, NumInterior, Colonia, Estado$$

3. PROVEEDOR

$$\texttt{PROVEEDOR}(IDProveedor, RazonSocial, Calle, NumExterior, NumInterior, Colonia, Estado)$$

Llave primaria:

$$IDProveedor$$

Dependencia funcional:

$$IDProveedor \rightarrow RazonSocial, Calle, NumExterior, NumInterior, Colonia, Estado$$

4. PERSONAL

$$\texttt{PERSONAL}(IDPersonal, IDSucursal, Nombre, ApellidoPaterno, ApellidoMaterno, CedulaProfesional, RFC, Calle, NumExterior, NumInterior, Colonia, Estado, Salario)$$

Llave primaria:

$$IDPersonal$$

Dependencia funcional principal:

$$IDPersonal \rightarrow IDSucursal, Nombre, ApellidoPaterno, ApellidoMaterno, CedulaProfesional, RFC, Calle, NumExterior, NumInterior, Colonia, Estado, Salario$$

5. CORREO_CLIENTE

$$\texttt{CORREO_CLIENTE}(IDCliente, CorreoElectronico)$$

Llave primaria compuesta:

$$(IDCliente, CorreoElectronico)$$

No hay dependencias funcionales no triviales adicionales. La llave solo evita que se repita el mismo correo para el mismo cliente.

No se debe asumir:

$$IDCliente \rightarrow CorreoElectronico$$

porque un cliente puede tener varios correos.

6. TELEFONO_CLIENTE

$$\texttt{TELEFONO_CLIENTE}(IDCliente, Telefono)$$

Llave primaria compuesta:

$$(IDCliente, Telefono)$$

No hay dependencias funcionales no triviales adicionales.
No se debe asumir:

$$IDCliente \rightarrow Telefono$$

porque un cliente puede tener varios teléfonos.

7. TELEFONO_PROVEEDOR

$$\texttt{TELEFONO_PROVEEDOR}(IDProveedor, Telefono)$$

Llave primaria compuesta:

$$(IDProveedor, Telefono)$$

No hay dependencias funcionales no triviales adicionales.
No se debe asumir:

$$IDProveedor \rightarrow Telefono$$

porque un proveedor puede tener varios teléfonos.

8. INSUMO

$$\begin{aligned}\texttt{INSUMO}(&NombreCientifico, Presentacion, FormaFarmaceutica,\\ &Concentracion, ViaAdministracion, Clasificacion, Descripcion,\\ &LaboratorioFabricante, NombreComercial, TipoDeControl,\\ &PrecioPublico, PrecioUnitario)\end{aligned}$$

Llave primaria:

$$NombreCientifico$$

Dependencia funcional:

$$\begin{aligned}NombreCientifico \rightarrow\ &Presentacion, FormaFarmaceutica, Concentracion,\\ &ViaAdministracion, Clasificacion, Descripcion, LaboratorioFabricante,\\ &NombreComercial, TipoDeControl, PrecioPublico, PrecioUnitario\end{aligned}$$

9. HORARIO_PERSONAL

$$\texttt{HORARIO_PERSONAL}(IDPersonal, Horario)$$

Llave primaria compuesta:

$$(IDPersonal, Horario)$$

No hay dependencias funcionales no triviales adicionales.
No se debe asumir:

$$IDPersonal \rightarrow Horario$$

porque un trabajador puede tener varios horarios.

10. CORREO_PERSONAL

$$\texttt{CORREO_PERSONAL}(IDPersonal, CorreoElectronico)$$

Llave primaria compuesta:

$$(IDPersonal, CorreoElectronico)$$

No hay dependencias funcionales no triviales adicionales.
No se debe asumir:

$$IDPersonal \rightarrow CorreoElectronico$$

porque un integrante del personal puede tener varios correos.

11. TELEFONO_PERSONAL

$$\texttt{TELEFONO_PERSONAL}(IDPersonal, Telefono)$$

Llave primaria compuesta:

$$(IDPersonal, Telefono)$$

No hay dependencias funcionales no triviales adicionales.
No se debe asumir:

$$IDPersonal \rightarrow Telefono$$

porque un integrante del personal puede tener varios teléfonos.

12. MEDICO

$$\texttt{MEDICO}(IDPersonal, Especialidad, InstitucionEgreso, VigenciaCertificacion)$$

Llave primaria:

$$IDPersonal$$

Dependencia funcional:

$$IDPersonal \rightarrow Especialidad, InstitucionEgreso, VigenciaCertificacion$$

13. ENFERMERA

$$\texttt{ENFERMERA}(IDPersonal, CertificadoReanimacion, TipoProcedimiento)$$

Llave primaria:

$$IDPersonal$$

Dependencia funcional:

$$IDPersonal \rightarrow CertificadoReanimacion, TipoProcedimiento$$

14. CAJERO

$$\texttt{CAJERO}(IDPersonal)$$

Llave primaria:

$$IDPersonal$$

No hay dependencias funcionales no triviales, porque la tabla solo contiene la llave.

15. LIMPIEZA

$$\texttt{LIMPIEZA}(IDPersonal)$$

Llave primaria:

$$IDPersonal$$

No hay dependencias funcionales no triviales.

16. CUIDADOR

$$\texttt{CUIDADOR}(IDPersonal)$$

Llave primaria:

$$IDPersonal$$

No hay dependencias funcionales no triviales.

17. FARMACEUTICO

$$\texttt{FARMACEUTICO}(IDPersonal)$$

Llave primaria:

$$IDPersonal$$

No hay dependencias funcionales no triviales.

18. CLINICA

$$\texttt{CLINICA}(IDClinica, IDSucursal, Nombre, NumCuartos)$$

Llave primaria:

$$IDClinica$$

Dependencia funcional:

$$IDClinica \rightarrow IDSucursal, Nombre, NumCuartos$$

19. HORARIO_CLINICA

$$\texttt{HORARIO_CLINICA}(IDClinica, Horario)$$

Llave primaria compuesta:

$$(IDClinica, Horario)$$

No hay dependencias funcionales no triviales adicionales.
No se debe asumir:

$$IDClinica \rightarrow Horario$$

porque una clínica puede tener varios horarios.

20. TELEFONO_SUCURSAL

$$\texttt{TELEFONO_SUCURSAL}(IDSucursal, Telefono)$$

Llave primaria compuesta:

$$(IDSucursal, Telefono)$$

No hay dependencias funcionales no triviales adicionales.
No se debe asumir:

$$IDSucursal \rightarrow Telefono$$

porque una sucursal puede tener varios teléfonos.

21. MEDICAMENTO

$$\texttt{MEDICAMENTO}(IDMedicamento, PrecioPublico, PrecioUnitario, MedicamentosEsteriles, Preparaciones, Formulacion, PreparadosOficiales, Pediatrica, Dermatologica, Stock)$$

Llave primaria:

$$IDMedicamento$$

Dependencia funcional:

$$IDMedicamento \rightarrow PrecioPublico, PrecioUnitario, MedicamentosEsteriles, Preparaciones, Formulacion, PreparadosOficiales, Pediatrica, Dermatologica, Stock$$

22. PROVEER_MEDICAMENTO

$$\texttt{PROVEER_MEDICAMENTO}(IDProveedor, IDMedicamento, IDSucursal, CondicionDeAlmacenamiento, Cantidad, FechaDeRecibo, FechaDeCaducidad)$$

Llave primaria compuesta:

$$(IDProveedor, IDMedicamento, IDSucursal, FechaDeRecibo)$$

Dependencia funcional:

$$IDProveedor, IDMedicamento, IDSucursal, FechaDeRecibo \rightarrow CondicionDeAlmacenamiento, Cantidad, FechaDeCaducidad$$

Aquí `FechaDeRecibo` forma parte de la llave porque distingue los lotes recibidos.

23. PROVEER_INSUMO

$$\texttt{PROVEER_INSUMO}(IDProveedor, IDSucursal, NombreCientifico, CondicionDeAlmacenamiento, Cantidad, FechaDeRecibo, FechaDeCaducidad)$$

Llave primaria compuesta:

$$(IDProveedor, IDSucursal, NombreCientifico, FechaDeRecibo)$$

Dependencia funcional:

$$IDProveedor, IDSucursal, NombreCientifico, FechaDeRecibo \rightarrow CondicionDeAlmacenamiento, Cantidad, FechaDeCaducidad$$

24. PREPARAR

$$\texttt{PREPARAR}(IDMedicamento, IDPersonal, Cantidad)$$

Llave primaria compuesta:

$$(IDMedicamento, IDPersonal)$$

Dependencia funcional:

$$IDMedicamento, IDPersonal \rightarrow Cantidad$$

25. USAR

$$\texttt{USAR}(IDPersonal, NombreCientifico)$$

Llave primaria compuesta:

$$(IDPersonal, NombreCientifico)$$

No hay dependencias funcionales no triviales adicionales.

26. UTILIZAR

$$\texttt{UTILIZAR}(IDMedicamento, NombreCientifico)$$

Llave primaria compuesta:

$$(IDMedicamento, NombreCientifico)$$

No hay dependencias funcionales no triviales adicionales.
No se debe asumir:

$$IDMedicamento \rightarrow NombreCientifico$$

porque un medicamento puede utilizar varios insumos.
Tampoco se debe asumir:

$$NombreCientifico \rightarrow IDMedicamento$$

porque un insumo puede ser usado por varios medicamentos.

27. TICKET

$$\texttt{TICKET}(IDTicket, IDSucursal, IDCliente, Fecha, Hora, \\ PrecioBruto, PrecioNeto, DescuentoAplicado)$$

Llave primaria:

$$IDTicket$$

Dependencia funcional:

$$IDTicket \rightarrow IDSucursal, IDCliente, Fecha, Hora, \\ PrecioBruto, PrecioNeto, DescuentoAplicado$$

28. COMPRAR

$$\texttt{COMPRAR}(IDTicket, IDMedicamento, Cantidad)$$

Llave primaria compuesta:

$$(IDTicket, IDMedicamento)$$

Dependencia funcional:

$$IDTicket, IDMedicamento \rightarrow Cantidad$$

Esto significa que, dentro de un mismo ticket, un medicamento aparece una sola vez con una cantidad asociada.

29. CONSULTA

$$\texttt{CONSULTA}(IDConsulta, IDCliente, IDMedico, IDEnfermera, \\ IDClinica, IDTicket, Fecha, Hora, Diagnostico, CostoConsulta)$$

Llave primaria:

$$IDConsulta$$

Dependencia funcional principal:

$$IDConsulta \rightarrow IDCliente, IDMedico, IDEnfermera, IDClinica, IDTicket, \\ Fecha, Hora, Diagnostico, CostoConsulta$$

Además, el DDL declara que `IDTicket` es único en `CONSULTA`, para mantener una relación 1:1 entre ticket y consulta. Por eso también se tiene:

$$IDTicket \rightarrow IDConsulta, IDCliente, IDMedico, IDEnfermera, \\ IDClinica, Fecha, Hora, Diagnostico, CostoConsulta$$

30. RECETA_MEDICA

$$\texttt{RECETA_MEDICA}(NumeroReceta, FechaNacimiento, Peso, Talla, \\ Alergias, Diagnostico, Consultorio, Turno)$$

Llave primaria:

$$NumeroReceta$$

Dependencia funcional:

$$NumeroReceta \rightarrow FechaNacimiento, Peso, Talla, Alergias, \\ Diagnostico, Consultorio, Turno$$

31. PEDIR

$$\texttt{PEDIR}(NumeroReceta, IDMedicamento, Dosis, Frecuencia)$$

Llave primaria compuesta:

$$(NumeroReceta, IDMedicamento)$$

Dependencia funcional:

$$NumeroReceta, IDMedicamento \rightarrow Dosis, Frecuencia$$

32. GENERAR_CONSULTA_RECETA

$$\texttt{GENERAR_CONSULTA_RECETA}(IDConsulta, NumeroReceta)$$

Llave primaria compuesta:

$$(IDConsulta, NumeroReceta)$$

Pero además el DDL declara unicidad para `IDConsulta` y para `NumeroReceta`, con la intención de asegurar una relación 1:1 entre consulta y receta. Por lo tanto, las dependencias funcionales importantes son:

$$IDConsulta \rightarrow NumeroReceta$$

$$NumeroReceta \rightarrow IDConsulta$$

La dependencia de la llave compuesta:

$$IDConsulta, NumeroReceta \rightarrow IDConsulta, NumeroReceta$$

es trivial, por eso las dependencias realmente útiles son las dos anteriores.

Relaciones sin dependencias funcionales no triviales

Las siguientes relaciones no tienen dependencias funcionales no triviales adicionales, porque sus atributos forman únicamente una llave compuesta o solo contienen la llave:

$$\begin{gathered} CORREO_CLIENTE, \quad TELEFONO_CLIENTE, \quad TELEFONO_PROVEEDOR, \\ HORARIO_PERSONAL, \quad CORREO_PERSONAL, \quad TELEFONO_PERSONAL, \\ HORARIO_CLINICA, \quad TELEFONO_SUCURSAL, \quad CAJERO, \\ LIMPIEZA, \quad CUIDADOR, \quad FARMACEUTICO, \\ USAR, \quad UTILIZAR \end{gathered}$$

7. Implementación física en PostgreSQL

La implementación física del esquema se realizó sobre el manejador **PostgreSQL** y quedó contenida en el archivo `DDL.sql`. Este archivo traduce el modelo relacional a un conjunto de instrucciones del Lenguaje de Definición de Datos (DDL) que crean las tablas, las llaves primarias, las llaves foráneas y las restricciones de integridad necesarias para que la base de datos refleje fielmente las reglas de negocio del caso de uso *Una farmacia de otro mundo*.

El esquema físico está compuesto por **32 tablas** que se pueden agrupar en cuatro categorías: entidades fuertes, especializaciones de la entidad `PERSONAL`, tablas de atributos multivaluados y tablas que representan relaciones (binarias muchos–a–muchos, ternarias y relaciones uno–a–uno). La organización del archivo sigue un orden lógico que facilita su ejecución desde cero: primero se eliminan las tablas existentes, después se crean todas las tablas, posteriormente se agregan las restricciones de dominio, las llaves primarias y, por último, las llaves foráneas.

7.1. Estrategia de creación y limpieza del esquema

El archivo inicia con una instrucción `DROP TABLE IF EXISTS ... CASCADE` que elimina todas las tablas del esquema en caso de que ya existan de forma cascada, que como se vio en clase y laboratorios, una vez que borras al padre, se borra a todos sus hijos. Esta decisión permite que el script `DDL.sql` se pueda ejecutar varias veces seguidas sin generar errores por objetos duplicados. Note que aseguramos la integridad de los datos a la hora de eliminar en forma de cascada.

Luego entonces, las tablas se crean con instrucciones `CREATE TABLE` en las que, en una primera etapa, se declaran únicamente las columnas con sus tipos de dato. Las restricciones (NOT NULL, CHECK, UNIQUE, llaves primarias y llaves foráneas) se agregan después con `ALTER TABLE`.

7.2. Selección de tipos de dato

Para cada atributo se eligió un tipo de dato específico a su contexto y lo que íbamos a necesitar saber de ellos. Los identificadores de las entidades fuertes se definieron como `INT`. Los nombres, direcciones y demás textos de longitud acotada se modelaron con `VARCHAR` de tamaño adecuado a cada caso (por ejemplo, `VARCHAR(80)` para nombres, `VARCHAR(13)` para el RFC y `VARCHAR(8)` para la cédula profesional, etc.). Los campos que no sabíamos qué tan largos podían ser, como `Diagnostico`, `Preparaciones` o `Formulacion`, se definieron con el tipo `TEXT`.

Los valores monetarios se modelaron con `DECIMAL` para conservar la precisión exacta requerida en cálculos de dinero: `DECIMAL(10,2)` para precios individuales y `DECIMAL(12,2)` para los totales del ticket, donde puede acumularse un importe mayor, todo esto, de acuerdo a lo visto en el laboratorio con Ricardo. El porcentaje de descuento se definió como `DECIMAL(5,2)`, suficiente para representar valores entre 0 y 100 con dos decimales. Las fechas se modelaron con `DATE` y las horas con `TIME`. Por último, las características booleanas de clientes y medicamentos (por ejemplo, `EsClienteEnLinea`, `EsPaciente`, `MedicamentosEsteriles` o `Pediatrica`) se modelaron con el tipo `BOOLEAN`, que expresa de forma natural la presencia o ausencia de una propiedad.

7.3. Integridad de entidad: llaves primarias

Cada tabla cuenta con una llave primaria que garantiza que no existan registros duplicados y que cada fila pueda identificarse de manera única. La estrategia para definir las llaves primarias dependió del tipo de tabla:

- **Entidades fuertes.** Las tablas `CLIENTE`, `SUCURSAL`, `PROVEEDOR`, `PERSONAL`, `MEDICAMENTO`, `CLINICA`, `TICKET`, `CONSULTA` y `RECETA_MEDICA` utilizan una llave primaria simple sobre su identificador pro-

pio. La tabla `INSUMO` es un caso particular, pues utiliza el atributo natural `NombreCientifico` como llave primaria, esto pues sabemos que el nombre científico del insumo no puede ser nulo y, sobre todo, no se puede repetir.

- **Especializaciones de PERSONAL.** Las tablas `MEDICO`, `ENFERMERA`, `CAJERO`, `LIMPIEZA`, `CUIDADOR` y `FARMACEUTICO` usan `IDPersonal` como llave primaria. De esta forma, cada especialización comparte el mismo identificador de la entidad general, lo que asegura una correspondencia uno–a–uno entre un trabajador y su rol.

- **Atributos multivaluados.** Las tablas como `TELEFONO_CLIENTE`, `CORREO_CLIENTE`, `HORARIO_PERSONAL` o `TELEFONO_SUCURSAL` usan una llave primaria compuesta formada por el identificador de la entidad y el valor del atributo. Esto permite almacenar varios teléfonos, correos u horarios para una misma entidad, evitando al mismo tiempo que se repita el mismo valor para una misma entidad.

- **Relaciones muchos–a–muchos.** Las tablas `COMPRAR`, `PEDIR`, `PREPARAR`, `USAR` y `UTILIZAR` usan una llave primaria compuesta por los identificadores de las dos entidades que relacionan.

- **Relaciones ternarias.** Las tablas `PROVEER_MEDICAMENTO` y `PROVEER_INSUMO` utilizan una llave primaria compuesta que incluye, además de los participantes de la relación, la `FechaDeRecibo`. Esto permite distinguir distintos lotes entregados por un mismo proveedor a una misma sucursal en fechas diferentes.

7.4. Integridad de dominio: NOT NULL, CHECK, UNIQUE y DEFAULT

Para garantizar que los datos almacenados sean válidos y coherentes, se declararon distintas restricciones de dominio sobre las columnas:

- **NOT NULL.** Se aplicó a los atributos obligatorios, como el nombre y los apellidos de clientes y personal, la fecha de nacimiento del cliente, el RFC y el salario del personal, o el nombre de la sucursal. Con esto se evita registrar entidades incompletas.

- **CHECK.** Se utilizaron para validar reglas de dominio más específicas. Algunos ejemplos importantes son: la fecha de nacimiento no puede ser futura; el RFC debe tener exactamente 13 caracteres y la cédula profesional entre 7 y 8; el salario, los precios y el stock no pueden ser negativos; el número de cuartos de una clínica debe ser mayor que cero; la fecha de caducidad de un lote debe ser posterior a su fecha de recibo; el descuento de un ticket debe estar entre 0 y 100; y el precio neto del ticket no puede ser mayor que el precio bruto.

- **UNIQUE.** Se aplicó al atributo `Usuario` de `CLIENTE` y al `RFC` de `PERSONAL`, garantizando su unicidad natural. También se usó sobre `IDTicket` en `CONSULTA` y sobre `IDConsulta` y `NumeroReceta` en la tabla `GENERAR_CONSULTA_RECETA`, con el fin de reforzar las cardinalidades uno–a–uno entre consulta, ticket y receta.

- **DEFAULT.** Se definieron valores por defecto para campos booleanos (inicializados en `FALSE`), para las fechas y horas del ticket (`CURRENT_DATE` y `CURRENT_TIME`), para los importes del ticket (inicializados en 0) y para el stock inicial de un medicamento. Estos valores por defecto permiten que ciertos registros se inserten sin necesidad de especificar todos los campos, dejando que la lógica de los disparadores complete la información posteriormente.

7.5. Integridad referencial: llaves foráneas y acciones referenciales

Todas las relaciones entre tablas se implementaron mediante llaves foráneas con acciones referenciales explícitas en `ON DELETE` y `ON UPDATE`. La elección de cada acción se realizó según el significado de la relación dentro del negocio:

- **ON UPDATE CASCADE.** Se utilizó de forma generalizada, de modo que si cambia el identificador de una entidad referenciada, el cambio se propague automáticamente a las tablas que dependen de ella.
- **ON DELETE CASCADE.** Se aplicó a las tablas dependientes cuya existencia no tiene sentido sin la entidad principal. Por ejemplo, los teléfonos y correos de un cliente se eliminan junto con el cliente, pues una vez que el cliente no está más, no interesa guardar su información asociada, mismo caso con las especializaciones que se eliminan junto con el personal, y los renglones de `COMPRAR` que se eliminan junto con su ticket.
- **ON DELETE RESTRICT.** Se aplicó a relaciones donde eliminar la entidad principal podría dejar inconsistente información valiosa. Por ejemplo, no se permite eliminar una sucursal mientras tenga personal asignado (pues pueden seguir perteneciendo a la farmacia aunque su sucursal tenga que cerrar), ni eliminar un medicamento mientras forme parte de una compra o de un suministro. Con esto se protege la historia operativa del sistema.
- **ON DELETE SET NULL.** Se aplicó en la relación entre `CONSULTA` y `ENFERMERA`, ya que la participación de la enfermera es opcional. Si se elimina el registro de una enfermera, la consulta se conserva, pero su referencia a la enfermera queda en nulo.

Un caso especial de integridad referencial es la referencia a roles especializados: la tabla `CONSULTA` no apunta directamente a `PERSONAL`, sino a `MEDICO` y `ENFERMERA`; de igual forma, `PREPARAR` y `USAR` apuntan a `FARMACEUTICO`. Esto asegura, a nivel de base de datos, que solo personal con el rol adecuado pueda participar en cada operación.

7.6. Documentación del esquema y orden de carga

Finalmente, el archivo `DDL.sql` incluye instrucciones `COMMENT ON TABLE` y `COMMENT ON COLUMN` que documentan, dentro del propio diccionario de datos de PostgreSQL, el propósito de las tablas y de las columnas más relevantes.

El orden de carga recomendado para reconstruir la base de datos completa es ejecutar primero `DDL.sql` (estructura), después `Trigger.sql` (lógica automática), después `DML.sql` (datos) y, por último, `SP.sql` y `Funciones.sql` (funciones y procedimientos), para finalmente poder ejecutar las consultas de `DQL.sql`. Este orden garantiza que cada componente encuentre disponibles las estructuras de las que depende.

8. Poblamiento de la base de datos

El poblamiento de la base de datos se realizó con el propósito de validar el correcto funcionamiento del modelo relacional desarrollado para el sistema de clínica y farmacia. La información insertada busca representar escenarios reales de operación dentro de la cadena de sucursales médicas, permitiendo comprobar tanto la integridad de los datos como el comportamiento de las consultas, restricciones y relaciones definidas en el esquema.

El objetivo principal del poblamiento fue generar un conjunto de datos suficientemente amplio y variado para:

- Verificar la integridad referencial entre tablas.
- Validar restricciones de dominio, llaves primarias y llaves foráneas.
- Simular operaciones reales de una clínica y farmacia.
- Garantizar que las consultas desarrolladas en el DQL devolvieran resultados representativos.
- Evaluar relaciones entre clientes, personal, medicamentos, consultas y sucursales.

Se consideraron distintos escenarios de uso reales dentro del sistema. Se generaron datos para clientes, sucursales, personal médico, enfermería, farmacéuticos, proveedores, medicamentos, consultas y tickets de compra.

Los registros fueron distribuidos de manera que existieran relaciones válidas entre las entidades principales del modelo. Por ejemplo:

- Cada empleado pertenece a una sucursal.
- Las consultas están asociadas a médicos, enfermeras y clientes existentes.
- Los tickets de compra están relacionados con clientes y sucursales válidas.
- Los medicamentos y los insumos cuentan con proveedores asociados.

Además, se incorporaron datos suficientes para cubrir distintos casos de prueba solicitados en las consultas del proyecto.

Se poblaron múltiples sucursales distribuidas en distintos estados para simular una operación real a gran escala. También se incluyeron diferentes tipos de empleados especializados, como médicos, enfermeras, cajeros, farmacéuticos y personal de limpieza, con el fin de validar las especializaciones del modelo relacional.

En cuanto a los clientes, se consideraron distintos perfiles:

- Clientes físicos.
- Clientes en línea.
- Pacientes que reciben consultas.
- Clientes que únicamente realizan compras.

Esto permitió ejecutar consultas relacionadas con historiales médicos, compras, tickets y comportamiento de usuarios.

El poblamiento también fue diseñado para garantizar resultados válidos en las consultas desarrolladas. Por ejemplo:

- Se insertaron enfermeras con apellidos maternos que contienen la secuencia "llo" para validar búsquedas con operadores como `ILIKE 'llo'`.
- Se agregaron clientes con compras registradas pero sin consultas médicas para comprobar consultas con `NOT EXISTS`.
- Se registraron consultas médicas en fechas y horarios específicos para validar filtros temporales.
- Se añadieron múltiples tickets y compras de medicamentos para realizar cálculos de precios brutos, precios netos e ingresos totales.

- Se incorporaron suficientes médicos y empleados en distintas sucursales para probar agrupaciones y conteos.

Gracias a esto, todas las consultas pudieron ejecutarse correctamente y devolver información coherente con el modelo propuesto.

El poblamiento también permitió comprobar el funcionamiento de las restricciones implementadas en el DDL, incluyendo:

- Restricciones NOT NULL.
- Restricciones CHECK.
- Restricciones UNIQUE.
- Llaves primarias.
- Llaves foráneas.
- Acciones ON DELETE y ON UPDATE.

De esta manera se verificó que los datos almacenados mantienen consistencia y cumplen las reglas del negocio establecidas.

9. Procedimientos, funciones y disparadores

Además de la estructura de tablas y restricciones definida en `DDL.sql`, la base de datos incorpora una capa de *lógica programada* que se ejecuta dentro del propio manejador. Con ella podemos implementar cosas que no están al alcance de la base de datos y que son de mucho interés para el cliente el hecho de que sean automáticos y obtener resultados interesantes a partir de esto. Conviene distinguir los tres tipos de objetos, ya que cumplen funciones distintas. Una **función** recibe parámetros y regresa un valor; se utiliza dentro de consultas o de otros objetos para obtener un resultado. Un **procedimiento almacenado** agrupa una serie de operaciones que se ejecutan mediante la instrucción `CALL`, y se usa principalmente para automatizar tareas que modifican la base de datos. Un **disparador** es una rutina que se ejecuta de forma automática cuando ocurre un evento dado sobre una tabla, como una inserción, una actualización o una eliminación.

9.1. Funciones (archivo `Funciones.sql`)

El archivo `Funciones.sql` contiene dos funciones que resuelven necesidades concretas de consulta sobre la información de la farmacia.

La primera función, `reconoceredad`, recibe el identificador de un cliente y regresa su edad en años cumplidos. Internamente busca la fecha de nacimiento del cliente en la tabla `CLIENTE` y, si la encuentra, calcula la diferencia respecto a la fecha actual utilizando las funciones `AGE` y `DATE_PART` de PostgreSQL. Si el cliente no existe o no tiene fecha registrada, la función regresa un valor nulo, lo que permite manejar el caso de forma controlada. Esta función es útil para generar reportes por grupos de edad o para validar condiciones que dependen de la edad del cliente.

La segunda función, `calculaganancias`, recibe el nombre de una sucursal y calcula sus ganancias durante el año 2026. Para ello suma dos componentes: por un lado, la venta de medicamentos, obtenida al multiplicar la cantidad comprada por el precio público en cada renglón de los tickets de la sucursal; y por otro, los ingresos por consultas, obtenidos al sumar el costo de las consultas realizadas en las clínicas de esa sucursal. Ambas sumas se filtran por el año correspondiente y se utiliza la función `COALESCE` para

que el resultado sea cero cuando no existan ventas o consultas, evitando así valores nulos. Esta función ofrece una visión rápida del desempeño económico de cada sucursal y puede aplicarse de forma masiva sobre todas las sucursales en una sola consulta.

9.2. Procedimientos almacenados (archivo `SP.sql`)

El archivo `SP.sql` contiene dos procedimientos almacenados que automatizan operaciones frecuentes de mantenimiento de la información.

El primero, `sp_registrar_farmaceutico`, registra a un nuevo integrante del personal y, al mismo tiempo, lo da de alta como farmacéutico. Antes de insertar cualquier dato, el procedimiento realiza una serie de validaciones: comprueba que el nombre y los apellidos no estén vacíos y que solo contengan letras y espacios (mediante una expresión regular que admite vocales acentuadas y la letra ñ); verifica que la cédula tenga la longitud esperada, que el RFC tenga 13 caracteres y que el salario no sea negativo; y confirma que la sucursal indicada exista. Si alguna validación falla, el procedimiento detiene la operación con un mensaje de error claro. Una vez superadas las validaciones, calcula el siguiente identificador de personal y, para evitar que dos sesiones obtengan el mismo identificador al mismo tiempo, bloquea brevemente la tabla `PERSONAL`. Finalmente, inserta el registro en `PERSONAL` y enseguida el mismo identificador en `FARMACEUTICO`, respetando el orden de dependencia entre ambas tablas. Este procedimiento es un buen ejemplo de cómo una operación que en realidad afecta a dos tablas se encapsula en una sola llamada segura y validada.

El segundo procedimiento, `sp_eliminar_medicamento`, elimina un medicamento a partir de su identificador. Dado que el esquema utiliza llaves foráneas con restricciones, no es posible eliminar directamente un medicamento que aún tiene registros relacionados. Por ello, el procedimiento primero verifica que el medicamento exista y después elimina, en orden, las filas que lo referencian en las tablas `PROVEER_MEDICAMENTO`, `PREPARAR`, `UTILIZAR`, `COMPRAR` y `PEDIR`, y solo al final elimina el registro principal de `MEDICAMENTO`. Durante el proceso utiliza la instrucción `GET DIAGNOSTICS` para informar cuántas filas se eliminaron en cada tabla relacionada, lo que da trazabilidad a la operación. De esta manera, una eliminación que podría ser compleja y propensa a errores se realiza de forma ordenada y segura.

9.3. Archivo de triggers

El archivo `Trigger.sql` contiene la definición de los disparadores utilizados para automatizar ciertas operaciones dentro de la base de datos. En este proyecto, estos disparadores se utilizan principalmente para mantener la consistencia de la información relacionada con el inventario de medicamentos y con el cálculo de los importes de los tickets de compra.

Este archivo forma parte del conjunto de scripts SQL del proyecto y no funciona de manera aislada. Para que los triggers puedan ejecutarse correctamente, primero deben existir las tablas, columnas, llaves primarias y llaves foráneas sobre las que trabajan. Por esta razón, el archivo `Trigger.sql` depende directamente del archivo `DDL.sql`, ya que en este último se define la estructura de la base de datos. En particular, los triggers utilizan tablas como `MEDICAMENTO`, `PROVEER_MEDICAMENTO`, `PREPARAR`, `COMPRAR` y `TICKET`, además de columnas como `Stock`, `Cantidad`, `PrecioPublico`, `PrecioBruto`, `DescuentoAplicado` y `PrecioNeto`.

En términos generales, el archivo `Trigger.sql` cubre dos procesos principales. El primero es la actualización automática del stock de medicamentos cuando se registran entradas o salidas de inventario. El segundo es el cálculo automático de los valores económicos del ticket, incluyendo el precio bruto, el descuento aplicado y el precio neto. Estos procesos son relevantes porque corresponden a operaciones frecuentes dentro del sistema y porque afectan directamente la exactitud de la información almacenada.

9.4. Importancia de los triggers

El uso de triggers permite reducir la cantidad de tareas manuales que tendrían que realizarse desde una aplicación externa o directamente por el usuario. Por ejemplo, si cada vez que se compra un medicamento se tuviera que actualizar manualmente el campo `Stock` en la tabla `MEDICAMENTO`, existiría un mayor riesgo de errores. Podría olvidarse el descuento, duplicarse una operación o modificarse un valor incorrecto. Al utilizar triggers, esta lógica queda integrada dentro de la propia base de datos.

Esto también ayuda a mantener una mayor consistencia en la información. La consistencia es importante porque los datos de distintas tablas están relacionados entre sí. Una compra registrada en la tabla `COMPRAR` debe verse reflejada en el inventario de `MEDICAMENTO`. De la misma forma, los productos asociados a un ticket deben afectar el total almacenado en la tabla `TICKET`. Si estas operaciones no estuvieran automatizadas, sería posible que el sistema mostrara una compra realizada, pero que el inventario no se hubiera reducido, o que un ticket tuviera productos asociados pero un total incorrecto.

9.5. Control del stock de medicamentos

Una de las funciones principales del archivo `Trigger.sql` es mantener actualizado el inventario de medicamentos. Este inventario se representa mediante la columna `Stock` de la tabla `MEDICAMENTO`. Dicha columna indica cuántas unidades disponibles existen de cada medicamento registrado en el sistema.

El control del stock se realiza a partir de tres tablas relacionadas con movimientos de medicamentos: `PROVEER_MEDICAMENTO`, `PREPARAR` y `COMPRAR`. Cada una de estas tablas representa un tipo distinto de operación. La tabla `PROVEER_MEDICAMENTO` representa la entrada de medicamentos por parte de proveedores. La tabla `PREPARAR` representa la preparación de medicamentos por parte del personal farmacéutico. La tabla `COMPRAR` representa la salida de medicamentos cuando un cliente realiza una compra.

El trigger asociado a `PROVEER_MEDICAMENTO` se encarga de aumentar el inventario cuando un proveedor entrega medicamentos. Cuando se inserta un nuevo registro en esta tabla, la cantidad recibida se suma automáticamente al `Stock` del medicamento correspondiente. Esto permite que el inventario refleje las nuevas existencias sin necesidad de hacer una actualización manual adicional en la tabla `MEDICAMENTO`.

Además, este trigger también contempla los casos en los que un registro de proveedor se modifica o se elimina. Cuando se actualiza una entrega de medicamento, el sistema no simplemente suma la nueva cantidad, porque eso produciría un resultado incorrecto. Primero debe revertir el efecto del valor anterior y después aplicar el nuevo valor. Por ejemplo, si originalmente se registraron 20 unidades y después se corrige el registro a 30 unidades, el sistema debe quitar primero las 20 unidades anteriores y luego sumar las 30 nuevas. De esta manera, el resultado final del inventario será correcto.

En caso de eliminación, el trigger descuenta del inventario la cantidad que había sido registrada como entrada. Esto tiene sentido porque, si se elimina el registro de una entrega, esa entrada ya no debe formar parte del cálculo de existencias. Sin este comportamiento, el stock podría quedar inflado artificialmente, ya que seguiría considerando medicamentos que en realidad ya no están respaldados por un registro válido.

La tabla `PREPARAR` funciona de manera similar, pero representa un origen diferente de medicamento. En este caso, el medicamento no llega directamente de un proveedor, sino que es preparado por personal farmacéutico. Cuando se registra una preparación, el trigger suma la cantidad preparada al `Stock` del medicamento correspondiente. Esto permite representar que, después de la preparación, existen más unidades disponibles para su uso o venta.

También se manejan actualizaciones y eliminaciones sobre `PREPARAR`. Si se cambia la cantidad preparada o el medicamento asociado, el sistema ajusta el inventario quitando primero el efecto del registro anterior y aplicando después el nuevo. Si se elimina la preparación, se descuenta la cantidad que había

sido agregada al stock. Este comportamiento mantiene la coherencia entre la tabla de preparaciones y la tabla principal de medicamentos.

La tabla `COMPRAR` representa el caso contrario a las entradas de inventario, ya que registra medicamentos vendidos o comprados por clientes. Por eso, el trigger asociado a esta tabla disminuye el `Stock` cuando se inserta una nueva compra. Antes de realizar el descuento, el sistema verifica que exista suficiente inventario disponible. Si la cantidad solicitada es menor o igual al stock actual, la operación se permite y el stock se actualiza. Si no hay suficiente inventario, el trigger genera un error y evita que la compra se registre.

Esta validación es especialmente importante porque impide que el inventario quede en valores negativos. Un stock negativo indicaría que el sistema permitió vender más unidades de las que realmente existían, lo cual representaría una inconsistencia grave. Al bloquear la operación cuando no hay existencias suficientes, el trigger ayuda a proteger la integridad de la información.

El trigger de `COMPRAR` también considera el caso de actualizaciones. Si una compra ya registrada se modifica, el sistema debe manejar el cambio cuidadosamente. Primero devuelve al inventario la cantidad anterior, como si la compra original se deshiciera. Después verifica si existe stock suficiente para aplicar la nueva cantidad. Si la validación se cumple, descuenta la cantidad actualizada. Este proceso es necesario porque una actualización puede cambiar no solo la cantidad, sino también el medicamento o el ticket asociado.

Por ejemplo, si originalmente se compraron 2 unidades de un medicamento y después se cambia a 5 unidades, no basta con restar 5 directamente, ya que las 2 unidades originales ya habían sido descontadas. Lo correcto es devolver primero las 2 unidades y luego intentar descontar 5. Esto evita errores acumulados en el inventario.

Cuando se elimina una compra, el trigger devuelve al inventario la cantidad que había sido descontada. Esto significa que, si la venta deja de existir dentro del sistema, las unidades correspondientes vuelven a estar disponibles. Este comportamiento es coherente con la idea de que el stock debe reflejar únicamente las compras actualmente registradas.

9.6. Cálculo de descuentos

Además del control de inventario, el archivo `Trigger.sql` tiene lógica para calcular descuentos asociados a los tickets. Esta parte se relaciona con la tabla `TICKET` y con la función `fn_calcular_descuento`. El objetivo de esta función es determinar qué porcentaje de descuento debe aplicarse a un ticket dependiendo de la cantidad de tickets registrados previamente durante ese mismo año.

La regla de descuento se basa en rangos. Si existen de 0 a 99 tickets previos en el año, el descuento aplicado es de `0` por ciento. Si existen de 100 a 499 tickets, el descuento es de `5` por ciento. Si existen de 500 a 1999 tickets, el descuento es de `10` por ciento. Entonces, si existen 2000 o más tickets previos, el descuento es de `15` por ciento.

Esta lógica permite que el sistema aplique descuentos de manera automática conforme aumenta la cantidad de operaciones registradas. El descuento no se calcula manualmente al momento de crear cada ticket, sino que se obtiene mediante una función que evalúa la cantidad de tickets existentes. Esto evita que el usuario tenga que recordar qué porcentaje corresponde a cada rango y reduce la posibilidad de asignar un descuento incorrecto.

El trigger encargado de aplicar este cálculo se ejecuta antes de insertar un nuevo registro en la tabla `TICKET`. Al ser un trigger de tipo `BEFORE INSERT`, puede modificar los valores del nuevo registro antes de que este quede guardado definitivamente. Este comportamiento es útil porque permite que el ticket sea insertado ya con su descuento inicial y con sus importes preparados.

Durante este proceso, el trigger revisa primero la fecha del ticket. Si la columna `Fecha` no tiene un valor definido, se asigna automáticamente la fecha actual mediante `CURRENT_DATE`. Después, con base en esa fecha, se cuentan los tickets existentes en el mismo año. A partir de ese número, se llama a la

función `fn_calcular_descuento` para obtener el porcentaje correspondiente y guardarlo en la columna `DescuentoAplicado`.

Además de calcular el descuento, este trigger inicializa los campos económicos del ticket. Si `PrecioBruto` o `PrecioNeto` no tienen valores definidos, se asigna un valor inicial de cero. Esto es importante porque, al momento de crear un ticket, puede que todavía no existan medicamentos asociados a él. En ese caso, el ticket puede iniciar con total cero y actualizarse posteriormente cuando se agreguen productos en la tabla `COMPRAR`.

9.7. Cálculo del precio bruto y precio neto

El archivo `Trigger.sql` también se encarga de mantener actualizados los importes del ticket. En particular, trabaja con los campos `PrecioBruto` y `PrecioNeto` de la tabla `TICKET`. El `PrecioBruto` representa el total antes de aplicar descuentos, mientras que el `PrecioNeto` representa el total final después de aplicar el porcentaje almacenado en `DescuentoAplicado`.

El cálculo del `PrecioBruto` se obtiene a partir de los medicamentos asociados al ticket en la tabla `COMPRAR`. Para cada medicamento comprado, se multiplica la cantidad registrada por el precio público del medicamento, almacenado en la columna `PrecioPublico` de la tabla `MEDICAMENTO`. Después, se suman todos esos resultados para obtener el total bruto del ticket.

Por ejemplo, si un ticket incluye dos medicamentos distintos, el sistema calcula el subtotal de cada uno multiplicando la cantidad por el precio correspondiente. Después suma ambos subtotales y guarda el resultado en `PrecioBruto`. Posteriormente, aplica el porcentaje de descuento guardado en `DescuentoAplicado` para obtener el `PrecioNeto`.

Este cálculo se actualiza automáticamente cuando ocurre un cambio en la tabla `COMPRAR`. Si se inserta una nueva compra asociada a un ticket, el total aumenta. Si se modifica una compra, el total se recalcula con los nuevos valores. Si se elimina una compra, el total disminuye. De esta forma, el ticket no depende de cálculos manuales ni de valores escritos directamente por el usuario.

Una ventaja de este enfoque es que el ticket siempre refleja el estado actual de sus productos asociados. Si cambia la cantidad de medicamentos comprados, el total se ajusta. Si se quita un medicamento del ticket, el total también se actualiza. Esto evita diferencias entre el detalle de la compra y los totales almacenados en la tabla `TICKET`.

El trigger también contempla el caso en el que una actualización mueva una compra de un ticket a otro. Esto puede ocurrir si se cambia el identificador del ticket en un registro de la tabla `COMPRAR`. En ese escenario, no basta con recalcular el ticket nuevo, porque el ticket anterior también queda afectado. Por eso, la lógica considera ambos casos y recalcula tanto el ticket que recibe la compra como el ticket del que se retiró.

Este comportamiento es importante porque evita que un ticket conserve un importe correspondiente a productos que ya no le pertenecen. También evita que el nuevo ticket quede sin considerar una compra que ahora sí forma parte de él. En otras palabras, el trigger mantiene sincronizada la relación entre el detalle de compra y los totales del ticket.

9.8. Eventos que detectan los triggers

Los triggers del archivo `Trigger.sql` consideran principalmente tres tipos de eventos: `INSERT`, `UPDATE` y `DELETE`. Cada uno de estos eventos tiene un efecto diferente sobre la información almacenada.

El evento `INSERT` ocurre cuando se agrega un nuevo registro. En el caso de `PROVEER_MEDICAMENTO` y `PREPARAR`, una inserción representa una entrada de inventario, por lo que el stock aumenta. En el caso de `COMPRAR`, una inserción representa una salida de inventario, por lo que el stock disminuye. En el caso

de `TICKET`, una inserción representa la creación de un nuevo ticket, por lo que se calculan sus valores iniciales.

El evento `UPDATE` ocurre cuando se modifica un registro existente. Este evento requiere más cuidado porque el sistema debe considerar tanto el valor anterior como el nuevo valor. Por eso se utilizan referencias a los datos previos y actuales del registro. La idea general es deshacer primero el efecto del registro anterior y aplicar después el efecto del registro actualizado.

El evento `DELETE` ocurre cuando se elimina un registro. En este caso, el sistema debe revertir el efecto que ese registro tenía sobre otras tablas. Si se elimina una entrada de proveedor, se descuenta del inventario la cantidad que había sido agregada. Si se elimina una preparación, también se descuenta la cantidad correspondiente. Si se elimina una compra, se devuelve al inventario la cantidad que había sido vendida y se recalcula el total del ticket.

Considerar estos tres eventos hace que la lógica sea más completa. No solo se automatiza lo que sucede cuando se insertan datos nuevos, sino también lo que ocurre cuando esos datos se corrigen o se eliminan. Esto es importante porque, en un sistema real, los registros pueden modificarse por errores de captura, cancelaciones, ajustes administrativos o cambios en la operación.

9.9. Relación con la integridad de los datos

El archivo `Trigger.sql` contribuye a la integridad de los datos porque ayuda a que la información relacionada entre distintas tablas se mantenga coherente. La integridad no depende únicamente de llaves primarias y llaves foráneas. Aunque estas restricciones son importantes para asegurar que los registros se relacionen correctamente, no siempre son suficientes para mantener actualizados los valores derivados, como el stock o los totales de un ticket.

Por ejemplo, una llave foránea puede asegurar que una compra haga referencia a un medicamento existente, pero no impide por sí sola que se vendan más unidades de las disponibles. Para controlar ese tipo de regla se necesita una validación adicional, como la implementada en el trigger de `COMPRAR`. De manera similar, una llave foránea puede asegurar que una compra pertenezca a un ticket válido, pero no recalcula automáticamente el total de ese ticket. Para eso se requiere la lógica adicional definida en los triggers.

En este sentido, los triggers funcionan como una capa complementaria de control. Las restricciones estructurales garantizan que las relaciones básicas sean válidas, mientras que los triggers permiten aplicar reglas más específicas del negocio. Esta combinación ayuda a que la base de datos sea más robusta y menos propensa a inconsistencias.

9.10. Alcance del archivo

El archivo `Trigger.sql` cubre una parte importante de la lógica automática del sistema, especialmente en lo relacionado con medicamentos y tickets. Su alcance principal puede resumirse en dos áreas: el mantenimiento automático del inventario de medicamentos y el cálculo automático de los importes económicos del ticket.

En cuanto al inventario, el archivo permite registrar entradas de medicamentos por proveedor, entradas por preparación y salidas por compra. También considera modificaciones y eliminaciones, lo que hace que la lógica sea más completa que si solo se trabajara con inserciones. Además, incluye una validación para evitar que una compra deje el inventario en negativo.

En cuanto a los tickets, el archivo permite asignar descuentos automáticos según la cantidad de tickets previos en el año y recalcular los importes cuando cambian los medicamentos asociados a un ticket. Esto facilita que los campos `PrecioBruto`, `DescuentoAplicado` y `PrecioNeto` se mantengan actualizados sin intervención manual.

10. Consultas

10.1. Consulta xiv — Distribución de sucursales por estado

- Los estados con mayor número de sucursales representan las zonas de mayor presencia comercial de Una Farmacia de Otro Mundo.
- Si existe una alta concentración en pocos estados, la empresa podría depender demasiado de ciertos mercados regionales.
- Una distribución equilibrada indicaría una estrategia de cobertura territorial más sólida.

Query Query History

```sql
--  Información administrativa de las sucursales de
--  Una Farmacia de Otro Mundo.
-- ============================================================



-- Consulta xiv
-- Distribución geográfica de las sucursales: cuántas existen
-- y en qué estados se encuentran, con porcentaje del total
-- nacional.

SELECT
    S.Estado,
    COUNT(*) AS TotalSucursales,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS PorcentajeDelTotal
FROM SUCURSAL AS S
GROUP BY S.Estado
ORDER BY
    TotalSucursales DESC,
    S.Estado;
```

Data Output Messages Notifications

	estado character varying (80)	totalsucursales bigint	porcentajedeltotal numeric
1	Chiapas	11	1.10
2	Baja California	10	1.00
3	Baja California Sur	9	0.90
4	Coahuila	8	0.80
5	QuerÃ©taro	8	0.80
6	Tlaxcala	8	0.80
7	Chihuahua	7	0.70
8	Colima	7	0.70
9	Guerrero	7	0.70
10	San Luis PotosÃ	7	0.70
11	YucatÃ¡n	7	0.70
12	Durango	6	0.60
13	Hidalgo	6	0.60

Total rows: 852 Query complete 00:00:00.294

10.2. Consulta xv — Sucursales con clínica y capacidad médica

- El porcentaje de sucursales con clínica integrada refleja el nivel de diversificación hacia servicios médicos.
- Las clínicas con más cuartos probablemente atienden una mayor demanda de consultas.
- Horarios amplios o más consultorios pueden asociarse con sucursales de alta demanda.

Query Query History

```sql
-- Consulta xv
-- Sucursales que cuentan con clínica médica integrada,
-- mostrando el nombre de la clínica, número de cuartos
-- y los horarios de atención (agrupados).

SELECT

    S.IDSucursal,
    S.Nombre        AS NombreSucursal,
    S.Estado,
    CL.IDClinica,
    CL.Nombre       AS NombreClinica,
    CL.NumCuartos,
    STRING_AGG(DISTINCT HC.Horario, '; ' ORDER BY HC.Horario) AS HorariosClinica
FROM SUCURSAL AS S
INNER JOIN CLINICA AS CL
    ON S.IDSucursal = CL.IDSucursal
LEFT JOIN HORARIO_CLINICA AS HC
    ON CL.IDClinica = HC.IDClinica
GROUP BY
```

Data Output Messages Notifications

	idsucursal integer	nombresucursal character varying (120)	estado character varying (80)	idclinica integer	nombreclinica character varying (150)	numcuartos integer	horariosclinica text
1	1	Sucursal Los SuperDatos 0...	Baja California Sur	1	Shakshaft	12	Lun-Dom 08:00-2...
2	2	Sucursal Los SuperDatos 0...	Chiapas	2	Garrettson	12	Lun-Sab 07:00-19...
3	3	Sucursal Los SuperDatos 0...	Hidalgo	3	Laybourne	16	Lun-Dom 08:00-2...
4	4	Sucursal Los SuperDatos 0...	Hidalgo	4	Mosey	3	Lun-Sab 07:00-19...
5	5	Sucursal Los SuperDatos 0...	Coahuila	5	Bonnaire	9	Lun-Dom 08:00-2...
6	6	Sucursal Los SuperDatos 0...	YucatÃ¡n	6	Siderfin	9	Lun-Sab 07:00-19...
7	7	Sucursal Los SuperDatos 0...	Chiapas	7	Dixsee	5	Lun-Sab 07:00-19...
8	8	Sucursal Los SuperDatos 0...	Tlaxcala	8	Conerding	18	Lun-Dom 08:00-2...
9	9	Sucursal Los SuperDatos 0...	Tlaxcala	9	Crutchley	19	Lun-Dom 08:00-2...
10	10	Sucursal Los SuperDatos 0...	Coahuila	10	Roughley	11	Lun-Dom 08:00-2...
11	11	Sucursal Los SuperDatos 0...	Puebla	11	Richold	15	Lun-Sab 07:00-19...
12	12	Sucursal Los SuperDatos 0...	YucatÃ¡n	12	Brine	20	Lun-Dom 08:00-2...
13	13	Sucursal Los SuperDatos 0...	Ciudad de MÃ©xico	13	Attle	2	24 horas

10.3. Consulta xvi — Cantidad de empleados por sucursal

- Las sucursales con más empleados probablemente tienen mayor flujo de clientes o mayores operaciones.
- Una sucursal con muy poco personal podría enfrentar saturación o tiempos largos de atención.
- Permite detectar cuáles sucursales requieren expansión de personal.

Query Query History

```sql
-- Consulta xvi
-- Número total de empleados por sucursal y por tipo de puesto.
-- Se realiza una pivot manual sobre las especializaciones
-- (médico, enfermera, farmacéutico, cajero, limpieza, cuidador).

SELECT
    S.IDSucursal,
    S.Nombre AS NombreSucursal,
    COUNT(DISTINCT P.IDPersonal)  AS TotalEmpleados,
    COUNT(DISTINCT ME.IDPersonal) AS TotalMedicos,
    COUNT(DISTINCT EN.IDPersonal) AS TotalEnfermeras,
    COUNT(DISTINCT FA.IDPersonal) AS TotalFarmaceuticos,
    COUNT(DISTINCT CA.IDPersonal) AS TotalCajeros,
    COUNT(DISTINCT LI.IDPersonal) AS TotalLimpieza,
    COUNT(DISTINCT CU.IDPersonal) AS TotalCuidadores
FROM SUCURSAL AS S
LEFT JOIN PERSONAL      AS P  ON S.IDSucursal = P.IDSucursal
LEFT JOIN MEDICO        AS ME ON P.IDPersonal = ME.IDPersonal
LEFT JOIN ENFERMERA     AS EN ON P.IDPersonal = EN.IDPersonal
```

Data Output Messages Notifications

Showing rows: 1 to 1000

	idsucursal [PK] integer	nombresucursal character varying (120)	totalempleados bigint	totalmedicos bigint	totalenfermeras bigint	totalfarmaceuticos bigint	totalcajeros bigint	totallimpieza bigint	totalcuidadores bigint
1	1	Sucursal Los SuperDatos 0...	15	6	1	2	1	2	1
2	4	Sucursal Los SuperDatos 0...	14	5	1	2	1	3	1
3	6	Sucursal Los SuperDatos 0...	12	7	2	0	1	2	0
4	30001	Sucursal Manual 001	12	2	2	2	2	2	2
5	30002	Sucursal Manual 002	12	2	2	2	2	2	2
6	30003	Sucursal Manual 003	12	2	2	2	2	2	2
7	30004	Sucursal Manual 004	12	2	2	2	2	2	2
8	30005	Sucursal Manual 005	12	2	2	2	2	2	2
9	30006	Sucursal Manual 006	12	2	2	2	2	2	2
10	30007	Sucursal Manual 007	12	2	2	2	2	2	2
11	30008	Sucursal Manual 008	12	2	2	2	2	2	2
12	30009	Sucursal Manual 009	12	2	2	2	2	2	2
13	30010	Sucursal Manual 010	12	2	2	2	2	2	2

10.4. Consulta xvii — Salarios promedio por categoría de empleado

- Los puestos con mayores salarios reflejan áreas de especialización o mayor responsabilidad.
- Diferencias salariales muy marcadas pueden evidenciar jerarquías operativas importantes.
- Permite evaluar si existe equilibrio entre funciones médicas, administrativas y operativas.

Query Query History

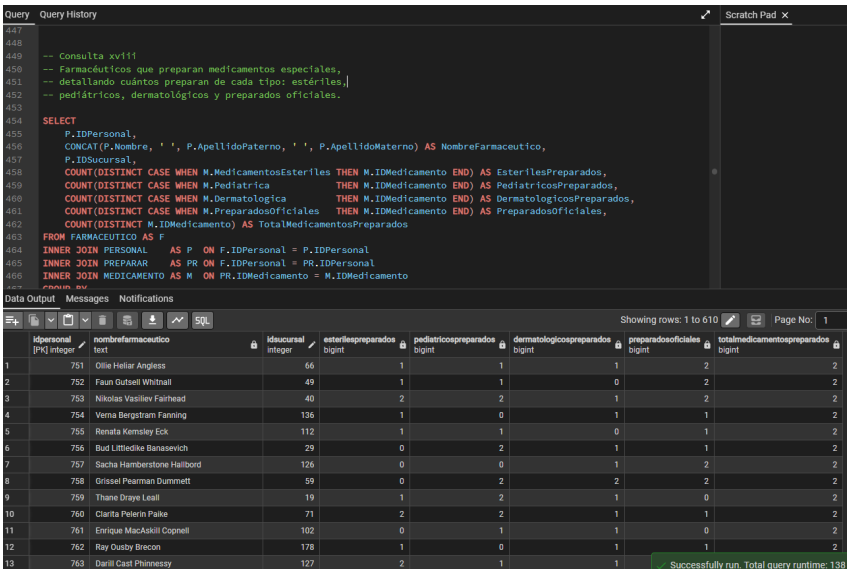
```sql
-- Consulta xvii
-- Salario promedio del personal por sucursal y por categoría
-- de puesto, incluyendo salario mínimo y máximo por grupo.

WITH PersonalCategorizado AS (
    SELECT
        P.IDPersonal,
        P.IDSucursal,
        P.Salario,
        CASE
            WHEN ME.IDPersonal IS NOT NULL THEN 'Medico'
            WHEN EN.IDPersonal IS NOT NULL THEN 'Enfermera'
            WHEN FA.IDPersonal IS NOT NULL THEN 'Farmaceutico'
            WHEN CA.IDPersonal IS NOT NULL THEN 'Cajero'
            WHEN LI.IDPersonal IS NOT NULL THEN 'Limpieza'
            WHEN CU.IDPersonal IS NOT NULL THEN 'Cuidador'
            ELSE 'Sin categoria'
        END AS Categoria
    FROM PERSONAL AS P
```

Data Output Messages Notifications

	idsucursal [PK] integer	nombresucursal character varying (120)	categoria text	totalpersonas bigint	salariopromedio numeric	salariominimo numeric	salariomaximo numeric
1	1	Sucursal Los SuperDatos 0...	Cajero	1	10063.86	10063.86	10063.86
2	1	Sucursal Los SuperDatos 0...	Cuidador	1	30950.20	30950.20	30950.20
3	1	Sucursal Los SuperDatos 0...	Enfermera	1	18100.00	18100.00	18100.00
4	1	Sucursal Los SuperDatos 0...	Farmaceuti...	2	27844.25	26546.84	29141.65
5	1	Sucursal Los SuperDatos 0...	Limpieza	2	23080.42	19579.38	26581.45
6	1	Sucursal Los SuperDatos 0...	Medico	6	32621.23	32150.00	33477.35
7	1	Sucursal Los SuperDatos 0...	Sin catego...	2	22343.10	16675.57	28010.62
8	2	Sucursal Los SuperDatos 0...	Cuidador	3	25744.69	21381.22	32878.21
9	2	Sucursal Los SuperDatos 0...	Enfermera	1	18200.00	18200.00	18200.00
10	2	Sucursal Los SuperDatos 0...	Farmaceuti...	1	26984.18	26984.18	26984.18
11	2	Sucursal Los SuperDatos 0...	Medico	5	33200.00	32900.00	33500.00
12	3	Sucursal Los SuperDatos 0...	Enfermera	2	13125.81	7951.61	18300.00
13	3	Sucursal Los SuperDatos 0...	Medico	5	33950.00	33650.00	34250.00

10.5. Consulta xviii — Farmacéuticos y preparación de medicamentos especiales

- Los farmacéuticos con mayor número de preparaciones pueden considerarse personal altamente especializado.
- Una alta frecuencia de medicamentos especiales indica demanda médica compleja.
- Permite identificar áreas donde se requiere capacitación técnica adicional.



```
-- Consulta xviii
-- Farmacéuticos que preparan medicamentos especiales,
-- detallando cuántos preparan de cada tipo: estériles,
-- pediátricos, dermatológicos y preparados oficiales.

SELECT
    P.IDPersonal,
    CONCAT(P.Nombre, ' ', P.ApellidoPaterno, ' ', P.ApellidoMaterno) AS NombreFarmaceutico,
    P.IDSucursal,
    COUNT(DISTINCT CASE WHEN M.MedicamentosEsteriles THEN M.IDMedicamento END) AS EsterilesPreparados,
    COUNT(DISTINCT CASE WHEN M.Pediatrica            THEN M.IDMedicamento END) AS PediatricosPreparados,
    COUNT(DISTINCT CASE WHEN M.Dermatologica         THEN M.IDMedicamento END) AS DermatologicosPreparados,
    COUNT(DISTINCT CASE WHEN M.PreparadosOficiales   THEN M.IDMedicamento END) AS PreparadosOficiales,
    COUNT(DISTINCT M.IDMedicamento) AS TotalMedicamentosPreparados
FROM FARMACEUTICO AS F
INNER JOIN PERSONAL    AS P  ON F.IDPersonal = P.IDPersonal
INNER JOIN PREPARAR    AS PR ON F.IDPersonal = PR.IDPersonal
INNER JOIN MEDICAMENTO AS M  ON PR.IDMedicamento = M.IDMedicamento
```

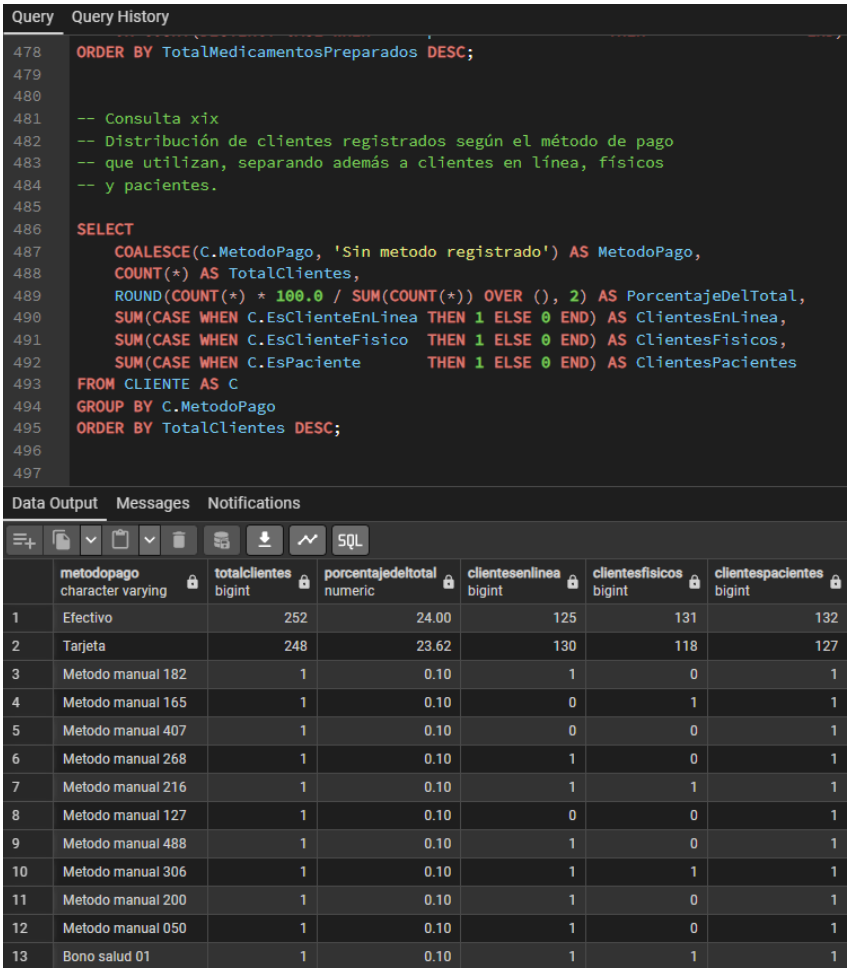
Showing rows: 1 to 610 — Page No: 1

	idpersonal [PK] integer	nombrefarmaceutico text	idsucursal integer	esterilespreparados bigint	pediatricospreparados bigint	dermatologicospreparados bigint	preparadosoficiales bigint	totalmedicamentospreparados bigint
1	751	Ollie Heliar Angless	66	1	1	1	2	2
2	752	Faun Gutsell Whitnall	49	1	1	0	2	2
3	753	Nikolas Vasiliev Fairhead	40	2	2	1	2	2
4	754	Verna Bergstram Fanning	136	1	0	1	1	2
5	755	Renata Kemsley Eck	112	1	1	0	1	2
6	756	Bud Littledike Banasevich	29	0	2	1	1	2
7	757	Sacha Hamberstone Hallbord	126	0	0	1	2	2
8	758	Grissel Pearman Dummett	59	0	2	2	2	2
9	759	Thane Draye Leall	19	1	2	1	0	2
10	760	Clarita Pelerin Paike	71	2	2	1	1	2
11	761	Enrique MacAskill Copnell	102	0	1	1	0	2
12	762	Ray Ousby Brecon	178	1	0	1	1	2
13	763	Darill Cast Phinnessy	127	2	1	1		

Successfully run. Total query runtime: 138

10.6. Consulta xix — Métodos de pago utilizados por clientes

- El método de pago más utilizado permite entender hábitos de consumo de los clientes.
- El predominio del efectivo podría indicar dependencia de clientes tradicionales.
- Ayuda a definir estrategias de promociones y plataformas de pago.

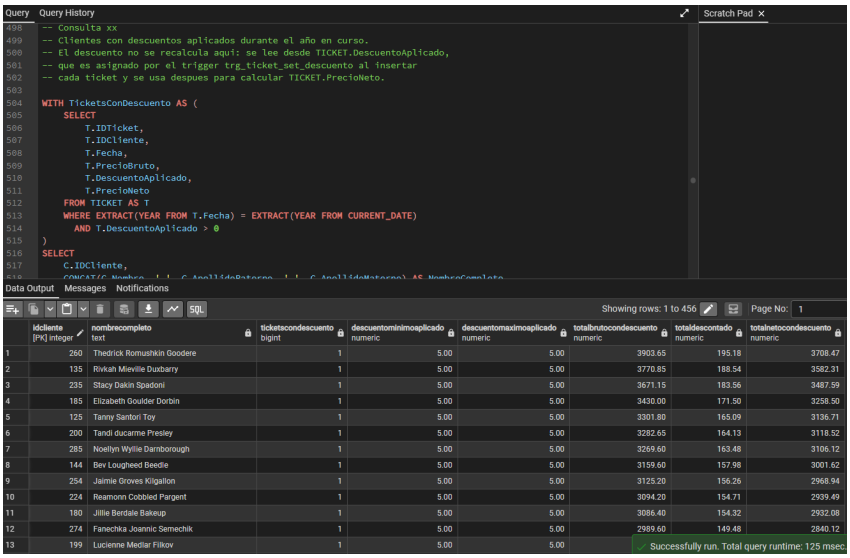


```
ORDER BY TotalMedicamentosPreparados DESC;


-- Consulta xix
-- Distribución de clientes registrados según el método de pago
-- que utilizan, separando además a clientes en línea, físicos
-- y pacientes.

SELECT
    COALESCE(C.MetodoPago, 'Sin metodo registrado') AS MetodoPago,
    COUNT(*) AS TotalClientes,
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS PorcentajeDelTotal,
    SUM(CASE WHEN C.EsClienteEnLinea THEN 1 ELSE 0 END) AS ClientesEnLinea,
    SUM(CASE WHEN C.EsClienteFisico  THEN 1 ELSE 0 END) AS ClientesFisicos,
    SUM(CASE WHEN C.EsPaciente       THEN 1 ELSE 0 END) AS ClientesPacientes
FROM CLIENTE AS C
GROUP BY C.MetodoPago
ORDER BY TotalClientes DESC;
```

	metodopago character varying	totalclientes bigint	porcentajedeltotal numeric	clientesenlinea bigint	clientesfisicos bigint	clientespacientes bigint
1	Efectivo	252	24.00	125	131	132
2	Tarjeta	248	23.62	130	118	127
3	Metodo manual 182	1	0.10	1	0	1
4	Metodo manual 165	1	0.10	0	1	1
5	Metodo manual 407	1	0.10	0	0	1
6	Metodo manual 268	1	0.10	1	0	1
7	Metodo manual 216	1	0.10	1	1	1
8	Metodo manual 127	1	0.10	0	0	1
9	Metodo manual 488	1	0.10	1	0	1
10	Metodo manual 306	1	0.10	1	1	1
11	Metodo manual 200	1	0.10	1	0	1
12	Metodo manual 050	1	0.10	1	0	1
13	Bono salud 01	1	0.10	1	1	1

10.7. Consulta xx — Tickets con descuentos o promociones

- Un alto número de descuentos puede indicar estrategias agresivas de fidelización.
- Clientes frecuentes con promociones recurrentes podrían representar consumidores leales.
- Si demasiadas ventas dependen de descuentos, podría existir riesgo de reducción de ganancias.
- Permite evaluar el impacto comercial de campañas promocionales.



```
-- Consulta xx
-- Clientes con descuentos aplicados durante el año en curso.
-- El descuento no se recalcula aqui: se lee desde TICKET.DescuentoAplicado,
-- que es asignado por el trigger trg_ticket_set_descuento al insertar
-- cada ticket y se usa despues para calcular TICKET.PrecioNeto.

WITH TicketsConDescuento AS (
    SELECT
        T.IDTicket,
        T.IDCliente,
        T.Fecha,
        T.PrecioBruto,
        T.DescuentoAplicado,
        T.PrecioNeto
    FROM TICKET AS T
    WHERE EXTRACT(YEAR FROM T.Fecha) = EXTRACT(YEAR FROM CURRENT_DATE)
      AND T.DescuentoAplicado > 0
)
SELECT
    C.IDCliente,
```

Showing rows: 1 to 456 — Page No: 1

	idcliente [PK] integer	nombrecompleto text	ticketscondescuento bigint	descuentominimoaplicado numeric	descuentomaximoaplicado numeric	totalbrutocondescuento numeric	totaldescontado numeric	totalnetocondescuento numeric
1	260	Thedrick Romushkin Goodere	1	5.00	5.00	3903.65	195.18	3708.47
2	135	Rivkah Mieville Duxbarry	1	5.00	5.00	3770.85	188.54	3582.31
3	235	Stacy Dakin Spadoni	1	5.00	5.00	3671.15	183.56	3487.59
4	185	Elizabeth Goulder Dorbin	1	5.00	5.00	3430.00	171.50	3258.50
5	125	Tanny Santori Toy	1	5.00	5.00	3301.80	165.09	3136.71
6	200	Tandi ducarme Presley	1	5.00	5.00	3282.65	164.13	3118.52
7	285	Noellyn Wyllie Darnborough	1	5.00	5.00	3269.60	163.48	3106.12
8	144	Bev Lougheed Beedle	1	5.00	5.00	3159.60	157.98	3001.62
9	254	Jaimie Groves Kilgallon	1	5.00	5.00	3125.20	156.26	2968.94
10	224	Reamonn Cobbled Pargent	1	5.00	5.00	3094.20	154.71	2939.49
11	180	Jillie Berdale Bakeup	1	5.00	5.00	3086.40	154.32	2932.08
12	274	Fanechka Joannic Semechik	1	5.00	5.00	2989.60	149.48	2840.12
13	199	Lucienne Mediar Filkov	1	5.00	5.00			

Successfully run. Total query runtime: 125 msec.

10.8. Consulta xxi — Número de consultas médicas realizadas

- Las sucursales con más consultas tienen mayor demanda de atención médica.
- Un incremento constante de consultas puede indicar confianza de los pacientes.
- Permite identificar horarios o temporadas de mayor carga médica.

postgres/postgres@PostgreSQL 18

Query Query History

```sql
-- Consulta xxi
-- Número de consultas médicas realizadas por mes,
-- mostrando además clientes distintos atendidos,
-- médicos activos, costo promedio e ingresos totales
-- por mes.

SELECT
    EXTRACT(YEAR  FROM CON.Fecha) AS Anio,
    EXTRACT(MONTH FROM CON.Fecha) AS Mes,
    TO_CHAR(CON.Fecha, 'YYYY-MM') AS PeriodoYM,
    COUNT(*)                      AS TotalConsultas,
    COUNT(DISTINCT CON.IDCliente)   AS ClientesDistintos,
    COUNT(DISTINCT CON.IDMedico)    AS MedicosActivos,
    ROUND(AVG(CON.CostoConsulta), 2) AS CostoPromedio,
    COALESCE(SUM(CON.CostoConsulta), 0) AS IngresosTotalesPorConsultas
FROM CONSULTA AS CON
GROUP BY
    EXTRACT(YEAR  FROM CON.Fecha),
    EXTRACT(MONTH FROM CON.Fecha),
```

Data Output Messages Notifications

	anio numeric	mes numeric	periodoym text	totalconsultas bigint	clientesdistintos bigint	medicosactivos bigint	costopromedio numeric	ingresostotalesporconsultas numeric
1	2026	11	2026-11	10	10	10	865.50	8655.00
2	2026	10	2026-10	31	31	31	592.13	18356.00
3	2026	9	2026-09	42	42	42	696.21	29241.00
4	2026	8	2026-08	62	62	61	765.71	47474.00
5	2026	7	2026-07	62	62	61	597.68	37056.00
6	2026	6	2026-06	60	60	60	758.50	45510.00
7	2026	5	2026-05	69	59	69	491.73	33929.20
8	2026	4	2026-04	30	30	30	399.80	11994.00
9	2026	3	2026-03	31	31	31	436.40	13528.40
10	2026	2	2026-02	28	28	28	471.80	13210.40
11	2026	1	2026-01	31	31	31	507.20	15723.20
12	2025	12	2025-12	31	31	31	544.40	16876.40
13	2025	11	2025-11	30	30	30	581.00	17430.00

10.9. Consulta xxii — Médicos activos y productividad

- Comparar médicos activos con consultas realizadas ayuda a medir productividad médica.
- Un bajo número de consultas por médico podría señalar poca demanda o sobrecontratación.
- Ayuda a planear mejor la distribución de médicos entre sucursales.

Query Query History

```sql
-- Consulta xxii
-- Top 51 medicamentos más recetados en todo el sistema,
-- con el número de recetas distintas en las que aparecen
-- y un resumen de las dosis prescritas.

SELECT
    M.IDMedicamento,
    COUNT(PE.NumeroReceta)              AS VecesRecetado,
    COUNT(DISTINCT PE.NumeroReceta)     AS RecetasDistintas,
    STRING_AGG(DISTINCT PE.Dosis, '; ' ORDER BY PE.Dosis) AS DosisFrecuentes,
    M.PrecioPublico,
    M.Stock                              AS StockActual
FROM MEDICAMENTO AS M
INNER JOIN PEDIR AS PE
    ON M.IDMedicamento = PE.IDMedicamento
GROUP BY
    M.IDMedicamento,
    M.PrecioPublico,
    M.Stock
ORDER BY VecesRecetado DESC
```

Data Output Messages Notifications

	idmedicamento [PK] integer	vecesrecetado bigint	recetasdistintas bigint	dosisfrecuentes text	preciopublico numeric (10,2)	stockactual integer
1	2	1	1	2 capsulas	770.09	123
2	3	1	1	1 tableta	430.12	132
3	4	1	1	5 ml	741.62	158
4	5	1	1	2 capsulas	101.88	184
5	6	1	1	1 tableta	736.45	208
6	7	1	1	5 ml	252.94	224
7	8	1	1	2 capsulas	102.39	240
8	9	1	1	1 tableta	205.73	259
9	10	1	1	5 ml	175.19	275
10	11	1	1	2 capsulas	179.05	298
11	12	1	1	1 tableta	517.58	315
12	13	1	1	5 ml	453.02	332
13	14	1	1	2 capsulas	571.41	349

10.10. Consulta xxiii — Recetas relacionadas con alergias

- Detectar medicamentos recetados pese a alergias registradas permite identificar posibles riesgos clínicos.
- Un número bajo de incidencias reflejaría buenos controles médicos.
- Casos repetidos podrían señalar errores en validaciones del sistema.

Query Query History

```sql
-- Consulta xxiii
-- Pacientes con alergias registradas en alguna receta médica,
-- mostrando el diagnóstico asociado y los medicamentos que se
-- les recetaron pese a la alergia. Útil para auditoría clínica.

SELECT
    C.IDCliente,
    CONCAT(C.Nombre, ' ', C.ApellidoPaterno, ' ', C.ApellidoMaterno) AS NombrePaciente,
    R.NumeroReceta,
    R.Alergias,
    R.Diagnostico,
    STRING_AGG(DISTINCT PE.IDMedicamento::text, ', ' ORDER BY PE.IDMedicamento::text) AS MedicamentosRecetados,
    CON.Fecha AS FechaConsulta
FROM CLIENTE AS C
INNER JOIN CONSULTA AS CON
    ON C.IDCliente = CON.IDCliente
INNER JOIN GENERAR_CONSULTA_RECETA AS GCR
    ON CON.IDConsulta = GCR.IDConsulta
INNER JOIN RECETA_MEDICA AS R
```

Data Output Messages Notifications

Showing rows: 1

	idcliente integer	nombrepaciente text	numeroreceta integer	alergias text	diagnostico text	medicamentosrecetados text	fechaconsulta date
1	2	Lacee Dipple Margrie	2	Polen	Sintomas leves y observacion	2	2026-05-17
2	3	Anthia Elldred Bohling	3	Lactosa	Consulta preventiva	3	2026-05-18
3	4	Tore Fredson Philippou	4	Mariscos	Revision general sin complicacio...	4	2026-05-19
4	5	Halsy Georgievski Enevold	5	Penicilina	Seguimiento de tratamiento	5	2026-05-20
5	7	Olympie Patise Cunniffe	7	Polen	Consulta preventiva	7	2026-05-22
6	8	Sheffy Waycott Drage	8	Lactosa	Revision general sin complicacio...	8	2026-05-23
7	9	Celestia Van der Kruis Smy	9	Mariscos	Seguimiento de tratamiento	9	2026-05-24
8	10	Penn Gisborne Baiden	10	Penicilina	Sintomas leves y observacion	10	2026-05-25
9	12	Bruce Hunday Aleksidze	12	Polen	Revision general sin complicacio...	12	2026-05-27
10	13	Jaquenette Iacofo Wurst	13	Lactosa	Seguimiento de tratamiento	13	2026-05-28
11	14	Hailee Toms Hanselmann	14	Mariscos	Sintomas leves y observacion	14	2026-05-29
12	15	Dorthy Cardero McMaster	15	Penicilina	Consulta preventiva	15	2026-05-30
13	17	Tanner Lawrance Cryer	17	Polen	Seguimiento de tratamiento	17	2026-06

10.11. Consulta xxiv — Stock e inventario por sucursal

- Las sucursales con mayor stock probablemente atienden una demanda más alta.
- Inventarios demasiado grandes podrían generar costos innecesarios de almacenamiento.
- Inventarios bajos podrían provocar desabasto de medicamentos importantes.

Query Query History

```sql
-- Consulta xxiv
-- Stock estimado de medicamentos por sucursal.
-- Se calcula como (total suministrado a la sucursal) - (total vendido en ella),
-- considerando todos los lotes de PROVEER_MEDICAMENTO y las ventas de COMPRAR.

WITH SuministrosPorSucursal AS (
    SELECT
        IDSucursal,
        IDMedicamento,
        SUM(Cantidad) AS TotalSuministrado
    FROM PROVEER_MEDICAMENTO
    GROUP BY
        IDSucursal,
        IDMedicamento
),
VentasPorSucursal AS (
    SELECT
        T.IDSucursal,
        CO.IDMedicamento,
        SUM(CO.Cantidad) AS TotalVendido
```

Data Output Messages Notifications

	idsucursal integer	nombresucursal character varying (120)	idmedicamento integer	totalsuministrado bigint	totalvendido bigint	stockestimadosucursal bigint
1	1	Sucursal Los SuperDatos 0...	2001	37	0	37
2	1	Sucursal Los SuperDatos 0...	2002	39	0	39
3	1	Sucursal Los SuperDatos 0...	2003	41	0	41
4	1	Sucursal Los SuperDatos 0...	2004	43	0	43
5	1	Sucursal Los SuperDatos 0...	2005	45	0	45
6	1	Sucursal Los SuperDatos 0...	2006	47	0	47
7	1	Sucursal Los SuperDatos 0...	2007	49	0	49
8	1	Sucursal Los SuperDatos 0...	2008	51	0	51
9	1	Sucursal Los SuperDatos 0...	2009	53	0	53
10	1	Sucursal Los SuperDatos 0...	2010	55	0	55
11	1	Sucursal Los SuperDatos 0...	2011	57	0	57
12	1	Sucursal Los SuperDatos 0...	2012	59	0	59
13	1	Sucursal Los SuperDatos 0...	2013	61	0	61

10.12. Consulta xxv — Medicamentos próximos a caducar

- Una gran cantidad de medicamentos próximos a vencer representa pérdidas potenciales.
- Podría indicar mala rotación de inventario en ciertas sucursales.
- Permite priorizar promociones o redistribución de productos antes de su vencimiento.

Query Query History

```sql
-- Consulta xxv
-- Medicamentos próximos a caducar (en los siguientes 90 días)
-- por sucursal, lote y proveedor, con la cantidad en inventario
-- y los días que faltan para el vencimiento.

SELECT
    PM.IDSucursal,
    S.Nombre  AS NombreSucursal,
    PM.IDMedicamento,
    PM.IDProveedor,
    PR.RazonSocial AS Proveedor,
    PM.Cantidad,
    PM.FechaDeRecibo,
    PM.FechaDeCaducidad,
    (PM.FechaDeCaducidad - CURRENT_DATE) AS DiasParaCaducar
FROM PROVEER_MEDICAMENTO AS PM
INNER JOIN SUCURSAL  AS S  ON PM.IDSucursal = S.IDSucursal
INNER JOIN PROVEEDOR AS PR ON PM.IDProveedor = PR.IDProveedor
WHERE PM.FechaDeCaducidad BETWEEN CURRENT_DATE AND CURRENT_DATE + INTERVAL '90 days'
```

Data Output Messages Notifications

Showing rows: 1 to 50

	idsucursal integer	nombresucursal character varying (120)	idmedicamento integer	idproveedor integer	proveedor character varying (150)	cantidad integer	fechaderecibo date	fechadecaducidad date	diasparacaducar integer
1	12	Sucursal Los SuperDatos 0...	2061	13	Kwideo	17	2026-05-29	2026-06-09	11
2	2	Sucursal Los SuperDatos 0...	2051	3	Rhyzio	18	2026-05-29	2026-06-10	12
3	13	Sucursal Los SuperDatos 0...	2062	14	Twitterwire	21	2026-05-29	2026-06-12	14
4	14	Sucursal Los SuperDatos 0...	2063	15	Cogidoo	13	2026-05-29	2026-06-14	16
5	3	Sucursal Los SuperDatos 0...	2052	4	Digitube	14	2026-05-29	2026-06-16	18
6	15	Sucursal Los SuperDatos 0...	2064	16	Podcat	26	2026-05-29	2026-06-17	19
7	16	Sucursal Los SuperDatos 0...	2065	17	Bluejam	18	2026-05-29	2026-06-19	21
8	17	Sucursal Los SuperDatos 0...	2066	18	Meetz	15	2026-05-29	2026-06-21	23
9	4	Sucursal Los SuperDatos 0...	2053	5	Chatterpoint	22	2026-05-29	2026-06-23	25
10	18	Sucursal Los SuperDatos 0...	2067	19	Topicware	20	2026-05-29	2026-06-24	26
11	19	Sucursal Los SuperDatos 0...	2068	20	Skalith	16	2026-05-29	2026-06-26	28
12	20	Sucursal Los SuperDatos 0...	2069	21	Nlounge	11	2026-05-29	2026-06-28	30
13	5	Sucursal Los SuperDatos 0...	2054	6	Muxo	16	2026-05-29	2026-06-29	31

10.13. Consulta xxvi — Proveedores de medicamentos e insumos

- Tener varios proveedores para un mismo producto reduce riesgos de desabasto.
- Productos con pocos proveedores generan mayor dependencia comercial.
- Ayuda a evaluar estabilidad de la cadena de suministro.

Query Query History

```sql
-- Consulta xxvi
-- Medicamentos que son suministrados por más de un proveedor
-- (proveedores que compiten por el mismo producto), incluyendo
-- la cantidad total suministrada por cada proveedor y el
-- precio unitario promedio del medicamento.

WITH MedicamentosConMultiplesProveedores AS (
    SELECT
        IDMedicamento,
        COUNT(DISTINCT IDProveedor) AS NumProveedores
    FROM PROVEER_MEDICAMENTO
    GROUP BY IDMedicamento
    HAVING COUNT(DISTINCT IDProveedor) >= 2
)
SELECT
    PM.IDMedicamento,
    MCMP.NumProveedores,
    PR.IDProveedor,
    PR.RazonSocial,
    SUM(PM.Cantidad)              AS CantidadTotalSuministrada,
```

Data Output Messages Notifications

	idmedicamento integer	numproveedores bigint	idproveedor integer	razonsocial character varying (150)	cantidadtotalsuministrada bigint	preciounitariopromedio numeric
1	2001	2	1	Demivee	21	75.50
2	2001	2	2	Zazio	16	75.50
3	2002	2	1	Demivee	22	76.00
4	2002	2	2	Zazio	17	76.00
5	2003	2	1	Demivee	23	76.50
6	2003	2	2	Zazio	18	76.50
7	2004	2	1	Demivee	24	77.00
8	2004	2	2	Zazio	19	77.00
9	2005	2	1	Demivee	25	77.50
10	2005	2	2	Zazio	20	77.50
11	2006	2	1	Demivee	26	78.00
12	2006	2	2	Zazio	21	78.00
13	2007	2	1	Demivee	27	78.50

10.14. Consulta xxvii — Ventas presenciales vs ventas en línea

- Un aumento de ventas en línea refleja adaptación digital de la empresa.
- Las sucursales con más ventas presenciales podrían depender de atención tradicional.
- Comparar ambos canales permite medir la efectividad de la plataforma digital.

Query Query History

```sql
-- Consulta xxvii
-- Ventas presenciales contra ventas en línea por sucursal.
-- Se utiliza la naturaleza del cliente (EsClienteEnLinea /
-- EsClienteFisico) para clasificar cada ticket. Si un cliente
-- es ambos, el ticket se reporta en la categoría "Mixto".

SELECT
    S.IDSucursal,
    S.Nombre AS NombreSucursal,
    COUNT(CASE
            WHEN C.EsClienteEnLinea AND NOT C.EsClienteFisico THEN T.IDTicket
        END) AS TicketsEnLinea,
    COUNT(CASE
            WHEN C.EsClienteFisico AND NOT C.EsClienteEnLinea THEN T.IDTicket
        END) AS TicketsPresenciales,
    COUNT(CASE
            WHEN C.EsClienteEnLinea AND C.EsClienteFisico THEN T.IDTicket
        END) AS TicketsMixtos,
    ROUND(SUM(CASE
```

Data Output Messages Notifications

	idsucursal [PK] integer	nombresucursal character varying (120)	ticketsenlinea bigint	ticketspresenciales bigint	ticketsmixtos bigint	montoenlinea numeric	montopresencial numeric	montomixto numeric
1	1	Sucursal Los SuperDatos 0...	3	4	7	679.78	704.54	1902.31
2	2	Sucursal Los SuperDatos 0...	3	4	8	475.65	1732.21	2686.46
3	4	Sucursal Los SuperDatos 0...	2	6	4	0.00	1777.80	3114.24
4	3	Sucursal Los SuperDatos 0...	6	3	5	1705.16	408.61	4209.01
5	5	Sucursal Los SuperDatos 0...	2	2	4	290.36	1546.61	2436.75
6	6	Sucursal Los SuperDatos 0...	3	1	1	4691.29	262.27	704.54
7	7	Sucursal Los SuperDatos 0...	1	2	1	731.59	1297.82	526.64
8	8	Sucursal Los SuperDatos 0...	2	2	1	1548.82	996.35	1139.96
9	9	Sucursal Los SuperDatos 0...	2	1	4	1011.66	704.44	3441.17
10	10	Sucursal Los SuperDatos 0...	2	1	1	1936.18	731.59	448.08
11	30001	Sucursal Manual 001	0	0	0	0.00	0.00	0.00
12	30002	Sucursal Manual 002	3	0	0	301.86	0.00	0.00
13	30003	Sucursal Manual 003	0	3	0	0.00	155.50	0.00

10.15. Consulta xxviii — Comparación entre ingresos médicos y ventas de medicamentos

- Las sucursales con mayores ingresos médicos tienen un enfoque más clínico.
- Las sucursales donde predominan ventas farmacéuticas funcionan principalmente como puntos comerciales.
- Permite identificar qué modelo de negocio resulta más rentable, como ampliar consultorios o fortalecer el inventario farmacéutico.

Query Query History

```sql
-- Consulta xxviii
-- Comparación de ingresos por venta de medicamentos vs ingresos
-- por consultas médicas, agrupado por sucursal. Muestra el
-- porcentaje del ingreso total que proviene de ventas para
-- identificar el modelo de negocio dominante de cada sucursal.

WITH IngresosVentas AS (
    SELECT
        T.IDSucursal,
        SUM(T.PrecioNeto)          AS IngresosPorVentas,
        COUNT(DISTINCT T.IDTicket) AS NumeroTickets
    FROM TICKET AS T
    GROUP BY T.IDSucursal
),
IngresosConsultas AS (
    SELECT
        CL.IDSucursal,
        SUM(CON.CostoConsulta) AS IngresosPorConsultas,
        COUNT(CON.IDConsulta)  AS NumeroConsultas
```

Data Output Messages Notifications

Showing rows: 1 to 1000

	idsucursal [PK] integer	nombresucursal character varying (120)	ingresosporventas numeric	numerotickets bigint	ingresosporconsultas numeric	numeroconsultas bigint	ingresostotales numeric	porcentajeingresoventas numeric
1	4	Sucursal Los SuperDatos 0...	7060.43	20	11192.00	15	18252.43	38.68
2	3	Sucursal Los SuperDatos 0...	6753.15	19	9704.00	15	16457.15	41.03
3	2	Sucursal Los SuperDatos 0...	6416.09	20	8216.00	15	14632.09	43.85
4	1	Sucursal Los SuperDatos 0...	6447.06	20	6728.00	15	13175.06	48.93
5	6	Sucursal Los SuperDatos 0...	7906.55	8	1388.00	3	9294.55	85.07
6	5	Sucursal Los SuperDatos 0...	5488.11	9	3130.00	5	8618.11	63.68
7	8	Sucursal Los SuperDatos 0...	6977.20	8	1484.00	3	8461.20	82.46
8	20	Sucursal Los SuperDatos 0...	6673.82	2	1410.00	2	8083.82	82.56
9	9	Sucursal Los SuperDatos 0...	5974.50	8	1532.00	3	7506.50	79.59
10	105	Sucursal Los SuperDatos 1...	6207.30	2	1040.00	2	7247.30	85.65
11	10	Sucursal Los SuperDatos 0...	5638.24	7	1580.00	3	7218.24	78.11
12	125	Sucursal Los SuperDatos 1...	5246.51	2	1800.00	2	7046.51	74.46
13	140	Sucursal Los SuperDatos 1...	5476.82	2	1470.00	2	6946.82	

Successfully r

11. Conclusión

Con este proyecto se logró diseñar e implementar una base de datos para el caso de uso de *Una farmacia de otro mundo*, tomando en cuenta diferentes áreas de una cadena de farmacias, como sucursales, clientes, personal, proveedores, medicamentos, insumos, consultas médicas, recetas y tickets. A lo largo del desarrollo se buscó que la información estuviera organizada de forma clara y que las relaciones entre las tablas representaran de manera adecuada las reglas principales del negocio.

Durante el proyecto se aplicaron varios temas vistos en el curso. Primero se trabajó el análisis de requerimientos para identificar las entidades, atributos y relaciones más importantes del caso. Después se construyó el modelo Entidad–Relación y se tradujo al modelo relacional, cuidando el uso de llaves primarias, llaves foráneas y tablas intermedias cuando era necesario. También se implementó el modelo físico en PostgreSQL mediante instrucciones DDL, agregando restricciones para mantener la *integridad* de los datos.

Además, se pobló la base de datos con información suficiente para poder probar su funcionamiento y se realizaron consultas SQL que permiten obtener información útil sobre la administración de esta farmacia. Estas consultas ayudan a revisar aspectos como clientes registrados, medicamentos disponibles, proveedores, ventas, consultas médicas y actividad por sucursal. Así la base de datos no solo guarda información, sino que también permite analizarla y obtener reportes relevantes.

También se utilizaron procedimientos, funciones y disparadores para automatizar algunas tareas y reforzar reglas dentro de la base de datos. Sin embargo, estos elementos fueron solo una parte del proyecto, ya que el objetivo principal fue integrar todo el proceso de diseño, construcción, poblamiento y consulta de una base de datos funcional.

En general, el proyecto permitió reforzar la importancia de diseñar correctamente una base de datos antes de implementarla. Este trabajo ayudó a entender mejor cómo pasar de un problema descrito en lenguaje natural a una estructura organizada en tablas, relaciones y restricciones.

Como mejora futura, la base de datos podría conectarse con una aplicación web o móvil para que el personal de la farmacia pudiera usarla de forma más sencilla. También se podrían agregar más reglas de negocio, reportes administrativos y controles de seguridad. Aun así, la base desarrollada cumple con el objetivo principal del proyecto: representar de manera ordenada y funcional la información necesaria para administrar esta farmacia.